

codex-windows / 019f30cc-e61d-7d30-93e8-4b0f0a070bfb

Metadata

```
- Source: `codex-windows`  
- Kind: `codex`  
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-05T11-12-42-019f30cc-e61d-7d30-93e8-4b0f0a070bfb.jsonl`  
- SHA-256: `92a374d4532acdcd0be2a4b0c1225fc4d4c69afb235d3a1cf1a77a0bfd7f26d`  
- Source modified: `2026-07-05T14:11:30+00:00`  
- Imported at: `2026-07-05T16:48:16+00:00`  
- cli_version: `0.142.5`  
- cwd: `C:\Users\avidu\Projects\khelsutra-guru`  
- model_provider: `openai`  
- session_id: `019f30cc-e61d-7d30-93e8-4b0f0a070bfb`  
- source: `vscode`
```

Transcript

1. developer (2026-07-05T05:44:19.152Z)

```
<permissions instructions>  
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-  
full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled.  
Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason,  
commands will be rejected.  
</permissions instructions>  
<app-context>
```

Codex desktop context

- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files

- In the app, the model can display images and videos using standard Markdown image syntax: `![alt](url)`
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., `![alt](/absolute/path.png)`); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., `[label](https://example.com)`).

Workspace Dependencies

- For sheets, slides, and documents, call ``load_workspace_dependencies`` to find the bundled runtime and libraries.

Automations

- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the ``automation_update`` tool first, then follow its schema instead of writing raw automation directives by hand.

- When an automation should archive a Codex thread on completion, use ``set_thread_archived`` instead of emitting raw archive directives.

Thread Coordination

- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: ``create_thread``, ``fork_thread``, ``list_threads``, ``read_thread``, ``send_message_to_thread``, ``handoff_thread``,

`set\_thread\_pinned`, `set\_thread\_archived`, or `set\_thread\_title`.

- Only use `create\_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create\_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments

- Use the `::code-comment{...}` directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: `::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}`

```
</app-context>
<collaboration_mode># Collaboration Mode: Default
```

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different ``<collaboration_mode>...</collaboration_mode>`` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request\_user\_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

```
</collaboration_mode>
<apps_instructions>
```

Apps (Connectors)

Apps (Connectors) can be explicitly triggered in user messages in the format `[\${app-name}](app://{connector\_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.

An app is equivalent to a set of MCP tools within the `codex\_apps` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool\_search` tool. If `tool\_search` is available, the apps that are searchable by `tools\_search` will be listed by it.

Do not additionally call `list\_mcp\_resources` or `list\_mcp\_resource\_templates` for apps.

```
</apps_instructions>
<skills_instructions>
```

Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

Skill roots

- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`

- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.5/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.3/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.3/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.7/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/slack/0.1.2/skills`
- `r13` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the out (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex d (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into \$CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting ima (file: r2/pdf/SKILL.md)
- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, mod (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)
- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggering a (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from sc (file: r2/xlsx/SKILL.md)

- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.101652/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.101652/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.101652/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on `.docx`, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render\_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final doc (file: r13/documents/26.630.12135/skills/documents/SKILL.md)
- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a

release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)

- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whe (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)
- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or cancel (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group

meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)

- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)

- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and referenc (file: r11/google-docs/SKILL.md)

- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)

- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)

- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)

- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)

- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r13/pdf/26.630.12135/skills/pdf/SKILL.md)

- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r13/presentations/26.630.12135/skills/presentations/SKILL.md)

- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)

- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)

- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)

- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)

- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-creator/SKILL.md)

- slack:slack: Read Slack context, route to the right Slack workflow, and prepare or perform Slack writes that match the user's intent. (file: r12/slack/SKILL.md)

- slack:slack-channel-summarization: Summarize activity from one Slack channel and return a concise recap, post-ready update, or summary doc. (file: r12/slack-channel-summarization/SKILL.md)

- slack:slack-daily-digest: Create a daily Slack digest from selected channels or topics. Use when the user asks for a daily Slack recap or summary of today's Slack activity. (file: r12/slack-daily-digest/SKILL.md)

- slack:slack-notification-triage: Triage recent Slack activity into a priority queue or task list for the user. (file: r12/slack-notification-triage/SKILL.md)

- slack:slack-outgoing-message: Primary skill for composing, drafting, or refining any outbound Slack content. Use this whenever the task will require using `slack\_send\_message`, `slack\_send\_message\_draft`, or `slack\_create\_canvas`. Use `slack` to read or analyze Slack context; use this skill to produce the final outgoing message. (file: r12/slack-outgoing-message/SKILL.md)

- slack:slack-reply-drafting: Draft Slack replies from available context. Use when the user wants help finding messages that likely need a response and preparing reply drafts. (file: r12/slack-reply-drafting/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r13/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes \$template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed arti (file: r13/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `\$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
 - 1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
 - 2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
 - 3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
 - 4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
 - 5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
 - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
 - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
 - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
 - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
 - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
 - Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.

</skills_instructions>

<plugins_instructions>

Plugins

A plugin is a local bundle of skills, MCP servers, and apps.

How to use plugins

- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin\_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp\_\_server\_\_tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.

- Missing/blocked: If the user requests a plugin that does not have relevant callable capabilities for the task, say so briefly and continue with the best fallback.

</plugins_instructions>

2. user (2026-07-05T05:44:19.152Z)

AGENTS.md instructions

<INSTRUCTIONS>

Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading

****Always `git pull` a repo before starting to read it**** ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- ****Why:**** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- ****How (safe form):**** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- ****If it can't fast-forward**** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts

Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit ****concurrently****, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a ****squash-merge** silently absorbed the sibling's work******. Protocol ? do this EVERY time, not only when you suspect a collision:

- ****Branch from the REMOTE, explicitly:**** `git fetch origin` then `git checkout -b <name> origin/<base>` (e.g. `origin/master`). Never assume the current branch is the intended base.
- ****Re-verify right before `git add`/commit:**** `git branch --show-current` + `git log --oneline -1` (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- ****Stage explicit paths**** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked files can't ride into your commit.
- ****Serialize repo writes:**** don't start a repo-writing spawned/background task while you hold uncommitted work ? commit or push first. If one may be running, treat the tree + current branch as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living ****session handoff**** for each project and use it to resume context quickly across windows.

- ****Where it lives:****
 - If the project has an auto-memory dir (`~/Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a ****"? Start here**** entry at the top of that project's `MEMORY.md`.
 - Otherwise, keep a `SESSION\_HANDOFF.md` at the repo root.
- ****What it contains**** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
 1. ****You are here**** ? current state.
 2. ****Next steps / open threads.****

3. ****Ramp-up kit**** ? the exact files/docs to read to get current.
4. ****Key decisions**** ? so they aren't relitigated.
- ****At session start:**** if a handoff exists, read it before starting substantive work, and use it (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- ****Updating it:**** keep it current ****automatically**** ? refresh at meaningful checkpoints (a PR is pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a restart can ramp up without losing context. Do ****not**** rewrite it every turn ? only when something worth resuming from changed. The phrases ****"update the handoff"**** / ****"refresh the handoff"**** force an immediate full rewrite on demand.
- ****Keep it honest:**** it reflects real, shipped state ? never aspirational. (See each project's attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do ****not**** post ****LGTM**** or otherwise sign off on a pull request until I have personally ****verified**** the following ? never trust the PR description's claims, reproduce them locally:

1. ****Full test suite is green.**** Run the whole suite (e.g. ``run_all_tests.py` / `pytest``), not just the changed files.
2. ****Install any dependency needed**** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. ****Lint + type checks are clean**** ? ``ruff check .`` and ``mypy ...``, matching the repo's CI. Install ``ruff`/`mypy`` if absent.
4. ****Coverage stays up**** ? run with coverage and confirm it stays ****at or above the repo's floor**** (e.g. the CI ``--cov-fail-under`` value) and does ****not regress**** versus the base branch.
5. ****Review findings are resolved**** ? bugs and repo-convention/discipline issues addressed.

LGTM is a ***verified*** stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.

</INSTRUCTIONS>

<environment_context>

```
<cwd>C:\Users\avidu\Projects\khehsutra-guru</cwd>
<shell>powershell</shell>
<current_date>2026-07-05</current_date>
<timezone>Asia/Calcutta</timezone>
<filesystem><workspace_roots><root>C:\Users\avidu\Projects\khehsutra-guru</root></workspace_roots><permission_profile type="disabled"><file_system type="unrestricted" /></permission_profile></filesystem>
</environment_context>
```

3. user (2026-07-05T05:44:19.171Z)

Hello. Can you read the session handoffs, memories etc. and then sync all the repos to the remote head since there were changes made in between. Ramp up on the Alpha launch tracker project and prepare to review a fix for <https://github.com/avidullu/khehsutra/issues/154> . Also let me know a prioritized list of pending items and I will plan to resolve them

4. assistant (2026-07-05T05:44:37.697Z)

I'll use the GitHub orientation and code-review workflows here, plus the local memory/handoff convention. First I'm fast-forwarding the current checkout, then I'll discover the related repos and pull them the same safe way before reading their state.

5. assistant (2026-07-05T05:44:52.132Z)

The working directory itself is a project container rather than a Git checkout, so I'm switching to repo discovery under it and nearby project folders. I'll keep pulls fast-forward only and treat any non-FF or dirty-blocked repo as local/stale rather than trying to ?fix? it.

6. assistant (2026-07-05T05:45:02.272Z)

I found the main KhelSutra checkout plus several sibling repos and worktree-style folders. Next I'm pulling each checkout/worktree independently; any repo that is already on a feature branch will be fast-forwarded only on that branch, so we preserve the concurrent-work safety rules.

7. assistant (2026-07-05T05:45:17.860Z)

The primary KhelSutra app is currently on `fix/154-alpha-copy-blockers`, which lines up nicely with the issue you mentioned. I'm going to include the worktree folders too after checking how they're wired, then I'll read the handoff/memory files from the refreshed state.

8. assistant (2026-07-05T05:45:30.622Z)

The `wt-\*` folders look like snapshot/copy work dirs rather than active Git worktrees: Git doesn't recognize them as checkouts, so there's nothing safe to pull there. Pulling the five actual KhelSutra repos now.

9. assistant (2026-07-05T05:45:58.123Z)

All five actual repos are now fast-forward checked and clean. I'm reading the handoff/memory surface next, starting with the project-level files and then the detailed ramp-up docs they point to.

10. assistant (2026-07-05T05:46:08.883Z)

There's a real handoff in `badminton-highlight-indexer`; the app repo itself doesn't appear to have a checked-in handoff file, but the copied `wt-\*` folders do. I'll prioritize the live repo handoff, then use the latest snapshot handoffs only as supporting context where they clearly refer to Alpha launch work.

11. assistant (2026-07-05T05:46:19.485Z)

The live tracker handoff is a little stale about the current branch, but its ramp-up kit is still useful. I'm reading the active Alpha tracker readiness docs and the KhelSutra app's current next-steps/docs so the pending list is grounded in current files, not just yesterday's handoff.

12. assistant (2026-07-05T05:46:26.008Z)

The active tracker says A13 delivered a conditional-GO review: the main outstanding condition is exactly `khesutra` issue #154 plus an owner browser CUJ. I'm going to inspect the issue/branch/PR state now, then I'll give you the prioritized unresolved list.

13. assistant (2026-07-05T05:46:39.263Z)

I'm pulling the GitHub issue metadata with `gh` because the connector exposed PR-focused tools but not issue reads. Locally, I'll compare the fix branch against `origin/master` so the review prep is tied to the actual patch, not just the commit message.

14. assistant (2026-07-05T05:46:45.199Z)

PR #155 is open, clean, not draft, and all four remote checks are green. I'm reading its diff and local test setup now so I can tell you what I'd review first and what still needs verified before an actual LGTM.

15. assistant (2026-07-05T05:46:53.895Z)

The GitHub connector timed out during startup, so I'm falling back to `gh` plus the synced local checkout. That still gives us the live issue/PR metadata and exact diff without waiting on the app bridge.

16. assistant (2026-07-05T05:47:06.898Z)

The PR is one commit over `origin/master` and touches only copy/locales plus two React

components/tests. I'm checking the scripts now; I won't call it LGTM unless the local verification gate is actually reproduced.

17. assistant (2026-07-05T05:47:22.972Z)

The patch shape matches the issue: hosted wizard copy, upload-moment disclosure, and privacy wording across source/public locales. I'm going to run the local review gate for the touched repo now: typecheck, coverage, build, and site tests.

18. assistant (2026-07-05T05:47:50.350Z)

Local typecheck/build/site tests passed, and coverage completed successfully. I'm grabbing the coverage numbers and checking whether there's a lint command; then I'll decide whether the browser e2e can be reasonably reproduced locally too.

19. assistant (2026-07-05T05:48:01.690Z)

No lint script is visible at the repo root, so I'm checking the web config directly. Coverage did not leave the usual JSON summary file, but the run itself exited 0; I'll still try to extract the thresholds from config so the verification line is precise.

20. assistant (2026-07-05T05:48:07.612Z)

The coverage gate is configured at 92/87/84/92 and the coverage run exited cleanly, so it cleared the floor. I'm running the web browser e2e CI script as the last local sanity check; after that I'll summarize what's ready and what you still need to decide.

21. assistant (2026-07-05T05:49:10.751Z)

The web browser e2e suite passed locally too: 5 Playwright CUIs green. I'm doing one last status/diff sanity pass and then I'll hand you the review-ready summary plus the pending-item priority stack.

22. assistant (2026-07-05T05:49:37.985Z)

The active repos are clean after the local checks. I'm taking a quick pass over the `wt-\*` handoffs now; they're non-git snapshot dirs, so I'll treat them as historical context and keep the live tracker/docs as the source of truth.

23. assistant (2026-07-05T05:49:49.332Z)

The snapshot handoffs are duplicated copies of the same A7A9 handoff; useful, but now superseded by the merged tracker/review docs. I'm pulling the current open issue lists once so the pending stack I give you is not guessing from stale doc text.

24. assistant (2026-07-05T05:49:53.537Z)

The issue lists confirm the live blockers are mostly in `kheIsutra` and the engine; collector is clean on open issues. One last tiny check on `sports-obsreport`, then I'll wrap this into an ordered plan.

25. assistant (2026-07-05T05:50:48.821Z)

****Synced And Ramped****

All five live KheIsutra repos are fast-forward synced and clean:

- `badminton-highlight-indexer` `master` ? `64fa4a2`
- `kheIsutra` `fix/154-alpha-copy-blockers` ? `73d7266`
- `rally-annotator` `main` ? `ef9e088`
- `sports-data-collector` `master` ? `c5b24bc`
- `sports-obsreport` `main` ? `a0c7252`

The top-level `kheIsutra-guru` folder is not itself a Git repo. The `wt-\*` folders are non-git

snapshot dirs; their handoffs are duplicate/stale copies of the A7-A9 tracker handoff, so I treated the live docs as source of truth.

I read the live handoff/memory and tracker context, especially
[SESSION_HANDOFF.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/SESSION_HANDOFF.md), [CLAUDE.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/CLAUDE.md), [ALPHA_LAUNCH_READINESS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ALPHA_LAUNCH_READINESS.md),
[ALPHA_LAUNCH_REVIEW_2026-07-04.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md), and
[khelsutra/NEXT_STEPS.md](C:/Users/avidu/Projects/khelsutra-guru/khelsutra/NEXT_STEPS.md).

****Issue #154 Fix****

The fix is PR [#155](https://github.com/avidullu/khelsutra/pull/155), closing issue [#154](https://github.com/avidullu/khelsutra/issues/154). It is open, non-draft, mergeable, one commit over `origin/master`, and all GitHub checks are green.

Local review prep passed:

- `web`: `npm run typecheck`
- `web`: `npm run test:coverage` passed the configured `92/87/84/92` coverage floor
- `web`: `npm run build`
- `web`: `npm run e2e:ci` ? 5 Playwright CUJs passed
- `site`: `npm test` ? 95 passed
- No lint script is present in `web/package.json`, `site/package.json`, or `.github`.

Patch focus is exactly right: hosted wizard copy at [SetupWizard.tsx](C:/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/components/SetupWizard.tsx:62), upload disclosure at [IngestPanel.tsx](C:/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/components/IngestPanel.tsx:196), English disclosure/privacy strings at [common.json](C:/Users/avidu/Projects/khelsutra-guru/khelsutra/locales/en/common.json:85), and static privacy drift copy at [privacy.html](C:/Users/avidu/Projects/khelsutra-guru/khelsutra/site/public/privacy.html:148). I'm ready to do the actual review/LGTM pass.

****Pending Items****

Priority order I'd use:

1. Review and merge `khelsutra` PR [#155](https://github.com/avidullu/khelsutra/pull/155). This clears the A13 current-allowlist copy blocker.
2. Owner browser CUJ through Cloudflare Access: upload ? process ? select ? compile ? download, with a normal allowlisted browser. This closes the tracker's unchecked tester path.
3. Record the owner GO/NO-GO for the current 4-email alpha allowlist.
4. Explicitly record A11 posture: resolve WASB C3 or defer with ?no commercial model sharing / no owned-model claims? copy.
5. Ratify A10 ship-gate target and allowed claims: [engine #172](https://github.com/Khelsutra/badminton-highlight-indexer/issues/172).
6. Record A12 decision: defer or reopen P10/P11 corpus flywheel for alpha-plus.
7. Before wider invites: replace the over-privileged runtime service account with least privilege.
8. Before wider invites: fix retention/deletion story: [#301](https://github.com/Khelsutra/badminton-highlight-indexer/issues/301) and [#447](https://github.com/Khelsutra/badminton-highlight-indexer/issues/447).
9. Before wider invites: safety/security/ops hardening: [#332](https://github.com/Khelsutra/badminton-highlight-indexer/issues/332), [#443](https://github.com/Khelsutra/badminton-highlight-indexer/issues/443), [#471](https://github.com/Khelsutra/badminton-highlight-indexer/issues/471), [#326](https://github.com/Khelsutra/badminton-highlight-indexer/issues/326), [#328](https://github.com/Khelsutra/badminton-highlight-indexer/issues/328).
10. Later polish/perf/corpus: [#331](https://github.com/Khelsutra/badminton-highlight-indexer/issues/331), [#349](https://github.com/Khelsutra/badminton-highlight-indexer/issues/349), [#444](https://github.com/Khelsutra/badminton-highlight-indexer/issues/444), [#445](https://github.com/Khelsutra/badminton-highlight-indexer/issues/445), [#175](https://github.com/Khelsutra/badminton-highlight-indexer/issues/175), plus native review of machine-draft `kn/hi` copy before wider launch.

26. user (2026-07-05T05:50:49.124Z)

[avidullu/khelsutra#155](https://github.com/avidullu/khelsutra/pull/155) is out for review. Please post an LGTM when you deem a PR good for merge or else keep responding to the PR and wait for the author to reply back

27. assistant (2026-07-05T05:51:06.208Z)

I'll take that as permission to complete the review loop on PR #155 and post to GitHub if it clears. I already did most of the gate; I'm checking the CI workflow shape now so the LGTM is earned, not vibes in a trench coat.

28. assistant (2026-07-05T05:51:22.169Z)

The CI workflow has one extra browser-layout job beyond the checks I already ran, and a shell-script parse check under `site`. I'm reproducing those locally too; if they pass and I don't find a code/copy issue, I'll post the LGTM directly on PR #155.

29. assistant (2026-07-05T05:52:02.033Z)

The extra layout/render/all browser checks all passed locally, and the shell scripts parse cleanly. I'm doing the final head/status check now, then I'll post an approving LGTM review on the PR.

30. assistant (2026-07-05T05:52:06.909Z)

Everything still points at the same reviewed head, `73d7266`, with GitHub checks green and the branch mergeable. Posting the approving review now.

31. assistant (2026-07-05T05:52:20.506Z)

GitHub won't let the authenticated account submit an approving review because this PR is authored by the same account. I'm posting the same LGTM as a PR conversation comment instead, so the review signal is still on-thread.

32. assistant (2026-07-05T05:52:36.425Z)

Posted LGTM on PR #155 as a comment: [issuecomment-4885014286](https://github.com/avidullu/khelsutra/pull/155#issuecomment-4885014286).

GitHub blocked a formal approving review because the authenticated account is the PR author, so I recorded the sign-off in the PR conversation instead. I found no blocking issues after reproducing the local gate: web typecheck, coverage, build, Playwright CUIs, site tests, layout/render/all, and shell parse check all passed; remote checks are green and the PR is mergeable.

33. user (2026-07-05T07:04:25.431Z)

Can you take up https://github.com/avidullu/khelsutra/issues/157 and do a thorough job at it? Make sure to use visual tools to evaluate the rendering and wordings etc.

34. assistant (2026-07-05T07:04:46.123Z)

I'll take this as an implementation task for `khelsutra` issue #157. I'm using the GitHub workflow for issue context and the in-app/browser visual workflow for rendering checks, and I'll branch from `origin/master` so none of the #155 branch state leaks in.

35. assistant (2026-07-05T07:05:06.121Z)

Issue #157 is about the ingest panel's cloud/bucket mode: selected-file flow should replace the setup controls with status, the medium copy is confusing, and upload status should show file size plus a time estimate. I'm creating a fresh issue branch from `origin/master` now, then I'll

inspect the ingest tests/components and design the smallest product-shaped fix.

36. assistant (2026-07-05T07:05:25.349Z)

`master` already includes the #154 merge now, so this issue branch can build on the current copy-disclosure work without stacking. I'm branching from `origin/master` and pulling down the issue screenshot so I can compare the owner's marked-up concern with the actual UI.

37. assistant (2026-07-05T07:05:54.478Z)

The issue screenshot URL is private enough that a plain download gets GitHub's 404 page; I'll retry with the authenticated token. Meanwhile the code confirms the UX problem: the setup controls remain visible during upload, and the "store this video" wording is accurate to the backend but not to the user's mental model.

38. assistant (2026-07-05T07:06:02.406Z)

Good, the authenticated screenshot download worked. I'm going to look at it directly, then I'll make the UI state transitions match the four marked points instead of just renaming a couple of strings.

39. assistant (2026-07-05T07:07:19.839Z)

I've got the shape now. I'm going to make this a real state-flow fix: clearer "add this video" wording, medium-specific file-button labels, a focused upload/progress state that hides setup controls while work is active, and upload metadata with file size plus ETA.

40. assistant (2026-07-05T07:07:27.447Z)

I'm editing the upload state machine first so the UI can show honest progress, not just a percentage. It will carry total bytes, bytes sent, and a computed time-left estimate that the panel can render.

41. assistant (2026-07-05T07:07:42.587Z)

Now I'm reshaping the ingest panel itself: the add-video controls will be quiet while work is running, and the visible button/help copy will change when the selected destination is a bucket or Drive.

42. assistant (2026-07-05T07:08:42.881Z)

The controls now collapse while a job is active. I'm adding the visible upload details next: selected file, total size, uploaded bytes, percentage, and time left.

43. assistant (2026-07-05T07:08:56.240Z)

One thing I noticed while wiring the metadata: the existing file-size formatter renders "0" bytes as "1 MB", which would look silly at the start of an upload. I'm going to tighten that helper so progress text starts honestly at "0 MB".

44. assistant (2026-07-05T07:09:03.668Z)

Actually that formatter is deliberately tuned for oversized-video error copy, so I'll leave it alone and use a separate upload-progress formatter instead. Progress and rejection copy have different jobs.

45. assistant (2026-07-05T07:09:16.693Z)

Next I'm updating the user-facing copy in the catalogs. I'm keeping English authoritative and giving the other locales straightforward translations so parity/build checks stay honest.

46. assistant (2026-07-05T07:09:38.223Z)

I'm applying the same wording structure across all six source catalogs, then I'll regenerate the `site/public/locales` mirror mechanically.

47. assistant (2026-07-05T07:11:58.916Z)

I'm picking this back up from the handoff: the branch is already cut from `origin/master`, the issue screenshot has been checked, and the core UX changes are partly in place. I'll finish the locale/test work, then run the app through screenshots so we can judge the actual rendering instead of just trusting the diff.

48. assistant (2026-07-05T07:12:08.403Z)

The working tree only has the intended partial edits plus the downloaded issue screenshot. I'm going to patch the remaining translations and then add tests that exercise the visible wording and the in-progress status surface.

49. user (2026-07-05T07:12:09.265Z)

I added a comment on the issue, can you please incorporate it. hence the interrupt

50. assistant (2026-07-05T07:12:23.726Z)

Thanks for the interrupt; that's exactly the kind of thing worth catching before I lock in tests and screenshots. I'll pull the latest issue discussion now and adjust the implementation before continuing.

51. assistant (2026-07-05T07:12:28.096Z)

The new comment asks for a lighter pass on all the X-marked wording: simplify the page and make the one remaining note less eager. I'm downloading that screenshot so I can tune against the marked UI, not just the text description.

52. assistant (2026-07-05T07:12:46.710Z)

The screenshot is clear: the dense helper paragraphs are doing too much before the user has a real video. I'm going to trim the pre-ingest guidance, shorten the cloud cost and storage helper text, and make the demo/sample controls quieter while preserving the necessary safety disclosures.

53. assistant (2026-07-05T07:12:50.722Z)

I found the marked strings: the top `filter by length` primer, cloud cost explainer, storage helper, file hint, and the sample-query primer/support text. I'm making those more compact and will add assertions so the page doesn't drift back into the dense copy.

54. assistant (2026-07-05T07:13:22.454Z)

I'm also going to shift the bucket path helper from a long explanation into a plain field hint. The status area will carry detail once work starts, so the setup panel can stay sparse.

55. assistant (2026-07-05T07:13:27.454Z)

I'm about to edit the locale strings and a small bit of component behavior. The main shape: less preamble, shorter helper text, dynamic bucket/Drive upload labels, and the in-progress panel becomes the place for upload size/ETA.

56. assistant (2026-07-05T07:13:50.842Z)

The English copy is trimmed. I'm carrying the same shape through the source locales now so the hosted locale bundles stay consistent after regeneration.

57. assistant (2026-07-05T07:14:24.300Z)

The top and sample-query copy is now short in Spanish too; I'm applying the same reduction across the ingest strings and the other locales.

58. assistant (2026-07-05T07:14:40.754Z)

One of the Spanish strings had a slightly different spelling/wording than the handoff snippet, so I'm reading the exact block and patching against the current file.

59. assistant (2026-07-05T07:14:45.008Z)

I've got the exact current wording now; continuing with precise patches.

60. assistant (2026-07-05T07:15:11.507Z)

I'm keeping these translations intentionally plain rather than trying to preserve the longer explanatory style. The visual goal here is fewer words at the point of decision.

61. assistant (2026-07-05T07:15:50.170Z)

The source locale changes are halfway through. After the Hindi/Kannada pass, I'll run the locale copy script so the app and site bundles don't diverge.

62. assistant (2026-07-05T07:17:28.454Z)

The locale strings are now in the compact version. I'm checking JSON validity before syncing them into `site/public`, then I'll move to tests and screenshots.

63. assistant (2026-07-05T07:17:37.809Z)

That JSON check tripped over PowerShell backtick interpolation, not the files. I'm rerunning it with shell-safe quoting, then I'll regenerate the public locale copies.

64. assistant (2026-07-05T07:17:44.622Z)

JSON parses cleanly. I'm syncing the locale copies now, then I'll tighten the component tests around the new bucket label, collapsed busy state, and upload metadata.

65. assistant (2026-07-05T07:17:54.502Z)

Locale bundles are regenerated. I'm moving to test coverage now, starting with the ingest panel tests because they can pin the exact behavior the issue describes.

66. assistant (2026-07-05T07:18:07.733Z)

I'm checking how upload progress is emitted so the test can pause during the real uploading state instead of racing past it.

67. assistant (2026-07-05T07:18:37.950Z)

I'm adding one focused test for the issue's core flow: bucket selected, destination entered, upload starts, setup copy disappears, and the live status shows file size plus ETA.

68. assistant (2026-07-05T07:19:11.612Z)

The panel test now pauses on the second upload chunk so the UI has a real in-progress state to assert against. I'm adding a small hook assertion too, to keep the byte/ETA metadata contract pinned.

69. assistant (2026-07-05T07:19:22.680Z)

The key tests are in place. I'm running the focused Vitest files first so any TypeScript or async timing issues show up quickly before the broader suite.

70. assistant (2026-07-05T07:19:59.533Z)

The new test raced itself because the mock upload was still using a 95 MB part size, so a 2 MB file never reached a second chunk. I'm fixing the mock to support small chunks and rerunning.

71. assistant (2026-07-05T07:20:07.718Z)

The mock now chunks at 1 MB for that test. I'm rerunning the ingest panel tests alone to keep the feedback tight.

72. assistant (2026-07-05T07:20:19.943Z)

The ingest tests pass now, including the paused upload status case. I'm rerunning the focused group, then I'll move to typecheck/build and visual inspection.

73. assistant (2026-07-05T07:20:32.782Z)

Focused tests are green. Next up: TypeScript/build checks, then I'll spin the UI with mocked APIs and capture screenshots for the wording/rendering pass.

74. assistant (2026-07-05T07:20:45.702Z)

Typecheck and site tests are clean. I'm running the full web test suite and production build before the browser pass; that gives the screenshots a real built app to inspect.

75. assistant (2026-07-05T07:21:35.858Z)

The full web suite and production build are green. I'm starting a local preview now and using Playwright to capture desktop/mobile states for the simplified setup copy and the in-progress upload status.

76. assistant (2026-07-05T07:22:18.668Z)

The preview came up, but the inline Playwright script ran from the repo root where `playwright` isn't installed. I'm rerunning it from `web/`, keeping the screenshots in the repo root.

77. assistant (2026-07-05T07:23:03.126Z)

The screenshots were captured successfully. I'm opening them now and checking for the exact things the issue called out: simplified wording, changed bucket button, no demo clutter while work is active, and upload size/ETA placement.

78. assistant (2026-07-05T07:23:21.949Z)

The screenshots caught one extra UX issue worth fixing before I call this done: the fixed reel footer is still visible over the no-video/sample state, which adds density and even overlaps the upload area on desktop. I'm going to hide the build footer until a real video is loaded; the sample grid can remain exploratory, but it shouldn't look buildable.

79. assistant (2026-07-05T07:23:40.114Z)

I'm tightening the app shell rather than the footer component itself: footer tests still cover the footer in isolation, and the app now only mounts it when there's an actual `videoId`.

80. assistant (2026-07-05T07:23:52.551Z)

With the footer hidden in the no-video state, I'm updating the few app-level tests that were using the demo footer as an always-visible target. The footer's own controls remain covered directly.

81. assistant (2026-07-05T07:24:16.968Z)

The app-level expectations are adjusted. I'm rerunning the touched test slice before taking a

fresh visual pass, because this change affects the page layout.

82. assistant (2026-07-05T07:24:31.584Z)

The one failure is just the i18n scaffold test: it doesn't skip the first-run wizard, so the ingest panel button is intentionally not mounted there. I'm keeping that test focused on the visible shell/query/grid copy.

83. assistant (2026-07-05T07:24:38.461Z)

That adjustment is in. I'm rerunning the touched slice again; after that I'll rebuild and refresh the screenshots so the final visual check reflects the footer fix.

84. assistant (2026-07-05T07:24:52.883Z)

The touched tests are green. I'm rebuilding, then I'll regenerate the screenshots to confirm the footer overlap is gone.

85. assistant (2026-07-05T07:25:12.838Z)

Build/typecheck are clean after the footer fix. I'm refreshing the screenshots now; this pass should show the setup state without the fixed build footer crowding it.

86. assistant (2026-07-05T07:25:45.074Z)

The fresh screenshots are ready. I'm doing one more visual inspection pass, mainly checking that the setup panel breathes better and that the upload status still gives the size/ETA without the sample controls.

87. assistant (2026-07-05T07:25:55.422Z)

The refreshed screenshots look materially cleaner: no fixed build footer in the empty/sample state, and the upload state collapses to the status panel with file size and ETA. I'm doing one final full web test run after that shell change.

88. assistant (2026-07-05T07:27:13.497Z)

I've picked the interrupted thread back up. I'm going to sanity-check the current diff, clean only the visual artifacts we generated, then run the remaining broad verification before I commit and open the PR.

89. assistant (2026-07-05T07:27:21.057Z)

Tiny wrinkle: the saved working directory is the project container, but Git says the repo itself is one level deeper or elsewhere. I'm locating the actual checkout before touching anything else so the cleanup and commit land in the right place.

90. assistant (2026-07-05T07:27:27.184Z)

Found it: the active checkout is `C:\Users\avidu\Projects\khelsutra-guru\khelsutra`. I'm switching my commands there and checking the worktree contents now.

91. assistant (2026-07-05T07:27:31.553Z)

The tracked changes are scoped to the ingest UX, locale copies, and tests; the untracked files are only the screenshots/log from visual QA. I'm removing those generated artifacts before doing the final verification pass.

92. assistant (2026-07-05T07:27:40.414Z)

The images cleaned up, but the preview log is still locked, which means the local preview process from visual QA is likely still running. I'm finding and stopping just that process so we don't leave background state behind.

93. assistant (2026-07-05T07:27:43.945Z)

That is exactly the old Vite preview pair. I'm stopping those two PIDs and then deleting the log; no source files are involved.

94. assistant (2026-07-05T07:27:51.828Z)

Artifacts are gone. I'm checking the available scripts so the last pass matches the repo's own test surface instead of guessing names.

95. assistant (2026-07-05T07:27:56.105Z)

The web app has explicit coverage and e2e CI scripts, so I'm going to run both now. The earlier focused and full unit suites were green; this is the final "be boring, be sure" pass.

96. assistant (2026-07-05T07:28:15.986Z)

Coverage passed. Next I'm running the Playwright CI path, which also rebuilds and runs the contract smoke test, so we get both browser behavior and production build validation in one pass.

97. assistant (2026-07-05T07:29:13.464Z)

E2E is green too: 5 Playwright tests passed, with the existing large-chunk Vite warning. I'm doing one final diff/status review now, including any generated coverage artifacts, before staging only the intended files.

98. assistant (2026-07-05T07:29:42.098Z)

A concurrency hazard just showed up: the checkout's current branch changed underneath us to `feat/processing-queue-and-dedup`, and two unrelated API files are now modified. I'm going to avoid committing from this shared checkout, isolate only the #157 file changes into a fresh worktree from `origin/master`, and leave the sibling branch/state untouched.

99. assistant (2026-07-05T07:30:02.531Z)

The branch name changed, but `HEAD` is still exactly `origin/master`; the extra API edits are uncommitted and unrelated. I'm copying only the #157 path diff into a temp patch, then applying it in a new worktree branched explicitly from `origin/master`.

100. assistant (2026-07-05T07:30:14.332Z)

I'm creating the temp patch now and fetching `origin` so the new worktree starts from the remote head, per your repo-freshness rule.

101. assistant (2026-07-05T07:30:18.685Z)

Patch captured. I'm making a fresh `origin/master` worktree for the PR branch and applying only that patch there.

102. assistant (2026-07-05T07:30:23.744Z)

The isolated worktree is ready on `fix/157-ingest-ux-polish` at `origin/master`. Applying the #157 patch now; after that I'll rerun enough verification in the clean worktree to make sure nothing depended on the shared checkout.

103. assistant (2026-07-05T07:30:34.498Z)

The patch didn't apply cleanly in the fresh worktree, which usually means the temp diff was encoded or based in a way `git apply` dislikes. I'm inspecting the patch header and the fresh file state, then I'll use a safer path-specific restore/copy approach if needed.

104. assistant (2026-07-05T07:30:43.331Z)

The patch content itself is valid-looking, and the fresh worktree is clean at the right commit. I'm checking line endings/content mismatch now; if it's just whitespace encoding, I'll apply with whitespace-tolerant options.

105. assistant (2026-07-05T07:30:47.363Z)

Whitespace-tolerant ``git apply --check`` passes, so this is just line-ending sensitivity. I'm applying with the same options in the clean worktree.

106. assistant (2026-07-05T07:30:59.305Z)

The #157 changes are now in the isolated branch. I'm checking dependency presence there; if the new worktree lacks ``node_modules``, I'll install from the lockfiles before rerunning verification.

107. assistant (2026-07-05T07:31:08.455Z)

The isolated worktree is cleanly scoped but has no dependencies installed. I'm going to run ``npm ci`` in ``web`` and ``site``, then repeat the checks from this branch.

108. assistant (2026-07-05T07:31:24.329Z)

Dependencies are installed with no vulnerabilities reported. I'm rerunning the branch-local verification: web typecheck, web coverage, and site tests first; Playwright e2e will follow after these finish.

109. assistant (2026-07-05T07:31:42.405Z)

Branch-local typecheck, coverage, and site tests are green. Now I'm running the branch-local e2e CI suite again; this also gives us another production build check.

110. assistant (2026-07-05T07:32:45.489Z)

E2E passed again in the clean worktree. I'm checking coverage artifacts and final git status now, then I'll stage the explicit #157 paths and commit.

111. assistant (2026-07-05T07:32:57.663Z)

The clean worktree has only the intended tracked files, and no commits ahead of ``origin/master`` yet. There isn't a coverage directory produced by this Vitest config, so I'm relying on the successful ``test:coverage`` exit; next I'm checking whether the web package has an explicit lint command before commit.

112. assistant (2026-07-05T07:33:08.420Z)

No lint script or lint deps are defined in ``web`` or ``site``, so there's nothing repo-native to run beyond typecheck/build/tests. I'm doing the branch-safety checks now, then staging the explicit file list only.

113. assistant (2026-07-05T07:33:14.254Z)

Branch safety checks are clean: the PR branch is at ``origin/master`` with no extra commits, and the diff list is exactly the intended #157 files. I'm staging those paths explicitly now.

114. assistant (2026-07-05T07:33:26.090Z)

Staged. I'm checking the staged diff and unstaged status once more before committing, because that branch switch earlier deserves extra paranoia.

115. assistant (2026-07-05T07:33:29.907Z)

The staged set is clean, with no unstaged drift. I'm committing the #157 UX work now.

116. assistant (2026-07-05T07:33:38.861Z)

Commit landed. I'm verifying that the branch contains only this one commit over `origin/master`, then I'll push and open the PR.

117. assistant (2026-07-05T07:33:42.651Z)

The branch is one commit ahead and `gh` is authenticated. Pushing `fix/157-ingest-ux-polish` now.

118. assistant (2026-07-05T07:33:50.461Z)

Branch is pushed. I'm checking for the GitHub connector PR-create tool; if it isn't exposed cleanly, I'll use the authenticated `gh` fallback.

119. assistant (2026-07-05T07:33:57.924Z)

The GitHub connector is available, so I'm opening the PR through it as a draft with the verification notes included.

120. assistant (2026-07-05T07:34:10.962Z)

The connector timed out during startup, so I'm using the authenticated `gh` fallback with the same PR body.

121. assistant (2026-07-05T07:34:31.780Z)

PR is open as draft: `#158`. Before I wrap, I'm updating the project handoff with the real shipped state and the concurrency note, since this is exactly the sort of checkpoint future sessions need.

122. assistant (2026-07-05T07:34:42.864Z)

I don't see a project memory directory or repo handoff file in the usual locations, so I'm doing one more targeted search for existing handoff/memory files before deciding whether to add a root `SESSION\_HANDOFF.md`.

123. assistant (2026-07-05T07:35:04.524Z)

There are existing Claude-era handoffs for KhelSutra, including a synced `khelsutra-guru` copy. I'm reading those and will update the active handoff rather than inventing a new one.

124. assistant (2026-07-05T07:35:14.297Z)

The active `khelsutra-guru` handoff is the OneDrive memory copy. I'm going to update its top checkpoint to include #157/#158 and the shared-checkout branch switch that happened today.

125. assistant (2026-07-05T07:35:25.069Z)

I'm adding a compact checkpoint rather than rewriting the whole history: current-state gets a new top block for #157/#158, and the handoff's first screen will point future sessions to that PR and the remaining priority list.

126. assistant (2026-07-05T07:35:48.593Z)

Current-state is updated. I'm refreshing the handoff's first page now so the next session starts from #158 and doesn't miss the shared-checkout hazard.

127. assistant (2026-07-05T07:36:17.071Z)

The handoff now points to #158. I'm updating the memory index's top `Start here` label so it doesn't still look like #155 is the freshest item.

128. assistant (2026-07-05T07:36:49.470Z)

One more housekeeping item: the shared checkout still has the duplicate #157 working-tree edits from before I isolated the PR, plus the unrelated sibling API WIP. I'm removing only my duplicated #157 paths from that shared checkout so it's left with just the sibling edits.

129. assistant (2026-07-05T07:37:01.615Z)

The shared checkout is cleaned of my duplicate #157 files. I'm doing the last status/PR check now, including confirming the original checkout only shows the unrelated sibling WIP.

130. assistant (2026-07-05T07:37:27.523Z)

Done. I incorporated the new issue comment/screenshot into #157 and opened draft PR [#158](https://github.com/avidullu/khelsutra/pull/158): **Polish ingest upload UX for cloud media**.

What changed:

- Simplified the X-marked dense wording and made cloud-cost copy less eager.
- Reworked upload state so setup controls collapse into a focused progress panel.
- Added uploaded size + ETA during upload.
- Fixed the sample/no-video state so the Build reel footer no longer crowds/overlaps.
- Updated all 6 locale catalogs and site mirrored catalogs.

Verified:

- Visual QA with original + new issue screenshots and desktop/mobile Playwright renders.
- ``web`: `npm run typecheck`, `npm run test:coverage`, `npm run e2e:ci``
- ``site`: `npm test`` with 95 passing.
- GitHub CI on #158 is green: site, web, layout, Workers build.
- Existing Vite large chunk warning remains.

I also updated the handoff/current-state memory. One important safety note: the shared checkout had switched underneath us to ``feat/processing-queue-and-dedup``; I isolated #157 into ``wt-157-ingest-ux`` and left the shared checkout with only the unrelated sibling WIP in ``web/src/api/client.ts`` and ``web/src/api/types.ts``.

Prioritized pending items:

1. Review/merge #158 for #157.
2. Owner browser CUJ + GO/NO-GO for A13.
3. Least-privilege service account before wider invites.
4. Retention automation / over-promise cleanup plus processor-list and acceptable-use copy.
5. Native-speaker review for kn/hi machine-drafted #154/#157 strings.
6. Then #329 async compile, #471 durable state, revision suffix, and finding-B promotion.

131. user (2026-07-05T07:44:14.210Z)

Could you please attach a "Before this PR" and "After this PR" screenshots i the PR so that the change is clear

132. assistant (2026-07-05T07:44:30.727Z)

Absolutely. I'll generate matched before/after screenshots from ``origin/master`` and the PR branch, then post them on #158 so reviewers can see the difference without hunting through the diff.

133. assistant (2026-07-05T07:44:50.952Z)

I'm using the GitHub workflow here, and the PR branch is up to date. Next I'm creating a clean `?before?` worktree at ``origin/master`` so the screenshots are a fair comparison, not a stale local state.

134. assistant (2026-07-05T07:45:19.148Z)

I found the existing mock engine that serves the built SPA. I'll use that plus Playwright route

overrides so both screenshots use the same cloud-capable/storage-capable state and the same upload interaction.

135. assistant (2026-07-05T07:45:29.044Z)

The before worktree is now pinned to `origin/master` at the pre-PR commit `5ddb81b`. I'm installing/building that SPA and rebuilding the PR branch so the static assets match each side exactly.

136. assistant (2026-07-05T07:45:38.567Z)

The PR build is ready; before has dependencies installed. I'm building the before SPA now, then I'll capture the paired views.

137. assistant (2026-07-05T07:45:55.692Z)

Both builds are ready. I'm going to capture two matched states for each side: the setup screen and an upload-in-progress screen, since the PR changes both wording density and active-upload focus.

138. assistant (2026-07-05T07:47:32.311Z)

First capture attempt hit a selector ambiguity: both `?In the cloud?` and `?Cloud bucket?` match `/cloud/`. I'll tighten that selector and rerun.

139. assistant (2026-07-05T07:48:22.090Z)

Screenshots are generated. I'm visually checking them now before posting so I don't attach a misleading or blank comparison.

140. assistant (2026-07-05T07:48:38.503Z)

The setup screenshots look good. The first upload `before` viewport didn't show the progress area because the old UI leaves it below the dense setup controls, which is exactly the problem but not a clear attachment. I'm recapturing the upload pair as focused ingest-panel screenshots so the old-vs-new upload state is obvious.

141. assistant (2026-07-05T07:49:15.563Z)

The upload pair has been recaptured as focused panel screenshots. I'm checking the two replacements now.

142. assistant (2026-07-05T07:49:20.447Z)

The four images now show the change clearly: the before setup/copy density, the after simplification, and the before upload panel versus the focused after upload panel. I'm uploading them to GitHub-hosted image URLs and then I'll post a PR comment with the rendered images.

143. assistant (2026-07-05T07:49:28.087Z)

I'm making this a secret/unlisted gist rather than committing screenshots into the PR branch. GitHub can render the raw gist image URLs directly in the PR comment.

144. assistant (2026-07-05T07:49:50.859Z)

The simple `gh gist` path won't work because GitHub's gist CLI refuses binary PNG uploads. I'm going to use the GitHub web attachment flow instead so the images are real PR attachments rather than files in the code branch.

145. assistant (2026-07-05T07:49:57.312Z)

The available GitHub connector only exposes download for existing user image attachments, not upload. I'm checking whether the browser/Chrome attachment path is callable in this session; if

it isn't, I'll use the least-invasive fallback: host the images on a throwaway branch and embed raw GitHub URLs in the PR comment.

146. assistant (2026-07-05T07:50:24.389Z)

Since there's no binary attachment upload API exposed here, I'm using a clean asset branch as the image host and embedding the raw GitHub URLs in a PR comment. That keeps the PR diff itself untouched while making the images render inline on GitHub.

147. assistant (2026-07-05T07:50:40.996Z)

The asset branch is isolated from both the PR branch and the shared checkout. I'm copying only the four generated PNGs into a `docs/pr-screenshots/158/` folder, then pushing that branch with CI skipped.

148. assistant (2026-07-05T07:50:59.740Z)

Branch safety checks are clean for the asset branch too: it starts at `origin/master` and has no commits yet. I'm staging just the four PNGs and pushing.

149. assistant (2026-07-05T07:51:15.860Z)

The image-hosting branch is pushed. I'm verifying the raw image URLs resolve, then I'll add the comparison comment to #158.

150. assistant (2026-07-05T07:51:31.325Z)

The plain raw URLs return 404 outside an authenticated GitHub context because the repo is private. I'm switching the comment to GitHub `blob?..raw=1` URLs, which render for authenticated repo viewers on the PR page. First I'm verifying the files exist through the GitHub API.

151. assistant (2026-07-05T07:51:49.295Z)

The files are present on the asset branch. I'm posting a PR comment now with authenticated GitHub image URLs and clear Before/After labels.

152. assistant (2026-07-05T07:52:08.507Z)

The PR comment is posted. I'm adding one tiny memory note that the screenshots are attached and where they're hosted, so future sessions don't wonder what the asset branch is.

153. assistant (2026-07-05T07:52:29.404Z)

Memory is updated. I'm doing a final status check across the PR branch, asset branch, and shared checkout before I close this out.

154. assistant (2026-07-05T07:52:54.145Z)

Posted the before/after screenshots on PR #158 here:

[PR comment with screenshots](<https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885297269>)

I included two comparisons:

- Setup / wording density
- Upload in progress at 50%

The PR branch and screenshot asset branch are both clean. The shared checkout is still on the sibling `feat/processing-queue-and-dedup` branch, now ahead by 1, so I left it untouched.

155. user (2026-07-05T07:54:24.731Z)

it is still in draft?

156. assistant (2026-07-05T07:54:32.130Z)

I'll check the live PR state now rather than trusting the last local snapshot.

157. assistant (2026-07-05T07:54:38.312Z)

Yes, #158 is still open as a draft.

I can mark it ready for review if you want me to.

158. user (2026-07-05T07:56:43.896Z)

i think as soon as we pick the video file the convert gets submitted automatically, can you check/

159. assistant (2026-07-05T07:57:02.230Z)

Good catch. I'll verify it against the PR branch, both in code and in the running UI, because the upload flow can blur together ?upload starts? and ?index/process starts.?

160. assistant (2026-07-05T07:57:21.863Z)

I'm using the debug workflow for this one. The first thing I'm checking is whether file selection starts only the upload, or whether it also queues the ingest/indexing job without another user action.

161. assistant (2026-07-05T07:57:35.952Z)

The code path already shows the important bit: selecting a file calls `upload.start(file)`, and `useUpload` calls the ingest callback after upload completion. I'm reproducing it in-browser now so we can distinguish ?upload begins immediately? from ?/api/process is queued immediately.?

162. assistant (2026-07-05T07:57:55.157Z)

That repro script tripped Node's module-mode parser, not the app. I'm rerunning it wrapped as CommonJS.

163. assistant (2026-07-05T07:58:25.514Z)

Confirmed. Picking a file does start the whole flow automatically.

Observed browser call sequence after selecting `match.mp4`:

```
```text
POST /api/upload/init
PUT /api/upload/.../part/0
POST /api/upload/.../complete
POST /api/assets
POST /api/process
```
```

So it is not `/api/process` at the exact instant of selection, but selection is effectively the submit action: upload completes, then store, then ingest/process queues automatically. This behavior already existed before #158; the PR made it more visible by focusing the upload state.

My recommendation: for this UX pass, change it to ?choose file ? show selected file ? explicit Start upload/index button.?

164. user (2026-07-05T08:00:23.409Z)

sure, you with your recommendation and also I posted [<https://github.com/avidullu/khelsutra/pull>

/158#issuecomment-4885313589](https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885313589) on the PR, can you incorporate that as well?

165. assistant (2026-07-05T08:00:40.375Z)

Yep, I'll fold both in: make file pick a deliberate "selected file" state with an explicit start action, and pull in your PR comment's wording/layout feedback. First I'm syncing the PR branch and reading the exact comment so I'm not guessing.

166. assistant (2026-07-05T08:00:48.665Z)

Your PR comment says the video name should be shown at the indicated place in a larger font. I'm downloading that screenshot now so I can match the placement instead of inferring from text alone.

167. assistant (2026-07-05T08:01:10.943Z)

The attachment URL doesn't download cleanly from the shell because GitHub serves it through an authenticated web path, but the comment text itself is clear enough: the selected video name needs to be visible in the indicated area, larger than the small status helper. I'm going to implement that as a prominent selected-file header inside the ingest panel, then verify visually.

168. assistant (2026-07-05T08:01:18.464Z)

Small correction: `gh api` can reach the attachment, so I'm saving it through that path and viewing it after all. That will let me place the larger filename where the screenshot actually points.

169. assistant (2026-07-05T08:02:28.058Z)

I'll incorporate both pieces now: the explicit "start after file pick" flow you approved, and the larger visible video-name treatment from the PR comment. I've already confirmed the current behavior does auto-submit on file selection, so this change is a real UX correction rather than just copy polish.

170. user (2026-07-05T08:02:28.452Z)

Files mentioned by the user:

videonameastitle.png: C:/Users/avidu/Pictures/Screenshots/videonameastitle.png

My request for Codex:

this is the image

```
<image name=[Image #1] path="C:\Users\avidu\Pictures\Screenshots\videonameastitle.png">
</image>
```

171. assistant (2026-07-05T08:02:46.328Z)

Thanks, that screenshot makes the intent much clearer: the filename shouldn't be a tiny progress detail; it should become the visible video title up near the "Process a video" heading while upload/indexing is underway. I'll make that title treatment work with the explicit-start flow too, so the selected video is visible before the user commits to processing.

172. assistant (2026-07-05T08:02:52.390Z)

I'm going to treat the screenshot's arrow literally: promote the current/selected video name into the card header area, while keeping progress details lower down. That gives the user immediate orientation and avoids hiding the thing they just chose in a tiny status row.

173. assistant (2026-07-05T08:03:03.806Z)

I'm synced and reading the implementation points now. The main behavior change will live in the ingest panel, and then I'll follow the ripples through locale strings and tests so the new flow is intentional everywhere.

174. assistant (2026-07-05T08:03:14.366Z)

Small repo-shape note: this worktree's app lives under `site/`, so I'm switching the reads there. Nothing is dirty so far; I'll keep the edits tightly scoped to the ingest UI, locale files, and the affected tests.

175. assistant (2026-07-05T08:03:24.094Z)

The app source is in `web/`; `site/` holds the deploy wrapper and copied locale assets. I've got the right place now and will make the master locale changes in `locales/`, then regenerate the site copy rather than hand-editing the mirrors.

176. assistant (2026-07-05T08:03:33.682Z)

I found the tests that currently prove the old auto-start behavior, so they'll be rewritten to prove the new safety behavior: file selection only selects and shows the title; the upload chain starts only after the explicit button click.

177. assistant (2026-07-05T08:04:32.855Z)

Before editing: I'm adding a pending-file state rather than reusing the upload state. That keeps ?chosen? and ?uploading? separate, which is the clean boundary the UX needs here.

178. assistant (2026-07-05T08:05:10.209Z)

The code now has the separation in place. Next I'm adding the new localized strings and changing the old ?Upload video?? chooser labels to ?Choose video?? so the copy no longer implies the upload begins immediately.

179. assistant (2026-07-05T08:06:19.354Z)

The master locale catalogs are updated. I'm regenerating the deploy-side locale copies now so the app and public site stay in sync.

180. assistant (2026-07-05T08:06:29.756Z)

Locale mirrors are refreshed. I'm updating the tests next so they lock in the new behavior instead of accidentally preserving the old auto-submit path.

181. assistant (2026-07-05T08:06:52.956Z)

One important test nuance: I'm not just adding a button click to make old tests pass. I'm adding assertions that no upload/process request is made immediately after file selection, so the regression the user noticed stays caught.

182. assistant (2026-07-05T08:07:31.041Z)

I'm also using the tests to pin the title placement: after a file is picked, the filename must appear in the new large title area before any network upload starts.

183. assistant (2026-07-05T08:08:09.213Z)

The old ?Upload video?? labels are gone from source and copied locale catalogs. I'm doing a syntax/test pass now; that usually catches the tiny UI-state edges after a change like this.

184. assistant (2026-07-05T08:08:19.791Z)

The first slice is ready, so I'm running the focused component tests now. If they're clean, I'll move outward to typecheck/build and then use the browser to inspect the rendered states visually.

185. assistant (2026-07-05T08:08:41.905Z)

Focused tests are green. I'm widening the verification now: typecheck, the full web test suite, and the site locale/test side, because this touched both React behavior and copied localization assets.

186. assistant (2026-07-05T08:09:04.383Z)

The broader checks are green so far: web typecheck, full web tests, site tests, and whitespace diff check all passed. I'm going to build the app and then do a browser pass with screenshots for the selected-file and uploading states.

187. assistant (2026-07-05T08:09:27.586Z)

Build and coverage also passed. The build still has the existing Vite large-chunk warning, but no failure. I'm going to capture the actual coverage numbers from the generated summary so the PR note has something concrete, then I'll move into browser verification.

188. assistant (2026-07-05T08:09:36.224Z)

Coverage didn't emit the small summary JSON in this setup, but the coverage run itself exited cleanly. The working diff is compact and scoped: ingest panel, two tests, and synchronized locale catalogs.

189. assistant (2026-07-05T08:09:48.264Z)

The coverage floor is 92/87/84/92 in Vite config, and `npm run test:coverage` passed against that. I'm starting a local app server next and will use browser routing to simulate capabilities/upload responses for the visual pass.

190. assistant (2026-07-05T08:10:01.902Z)

I'm spinning up a local Vite server on a separate port for the visual pass. The browser will mock the API calls, so I can inspect the exact UI states without needing a real upload or engine job.

191. assistant (2026-07-05T08:10:26.184Z)

The server start command returned quietly, so I'm checking the port and logs before driving the browser. If it didn't bind, I'll restart it cleanly on another port.

192. assistant (2026-07-05T08:10:34.832Z)

The app is live at `http://127.0.0.1:5174/`. I'm connecting the in-app browser now and will capture the selected-file and in-flight upload views from the real rendered page.

193. assistant (2026-07-05T08:10:56.329Z)

The in-app browser API doesn't expose request routing directly, so I'll use the app's existing Vite proxy path and run a tiny local mock API on `127.0.0.1:8000` if that port is free. That keeps the browser pass realistic: the SPA still calls `/api/\*` just as it would in dev.

194. assistant (2026-07-05T08:11:14.869Z)

I'll keep the mock API deliberately slow on the upload part endpoint so the upload progress state is visible long enough to inspect. After screenshots, I'll close the local helper processes.

195. assistant (2026-07-05T08:12:09.667Z)

The in-app browser is connected, but its supported file-picker API doesn't include setting a file programmatically. I'm switching the file-selection portion to a headless Playwright screenshot script, which can exercise the real file input and capture the rendered states accurately.

196. assistant (2026-07-05T08:12:23.595Z)

I'm setting the first-run wizard as already completed in the visual run, matching the app state from the screenshot you shared. The screenshots will focus only on the ingest panel states we changed.

197. assistant (2026-07-05T08:12:53.636Z)

The first screenshot script hit Node's module-mode guard, so I'm rerunning it as an ES module. Same visual flow, just a cleaner invocation.

198. assistant (2026-07-05T08:14:31.082Z)

The visual run confirmed the important behavior mechanically: after selecting the file, no ``/api/upload`` or ``/api/process`` request was made. It also produced desktop/mobile screenshots; I'm opening those now to inspect spacing, title prominence, and wrapping.

199. assistant (2026-07-05T08:14:54.346Z)

The screenshots look good for the changed area: the filename is now a prominent title, the selected state has an explicit start button, and the upload state keeps that title while progress moves below it. I'm checking the Playwright CUIs now because a file-upload behavior change can easily break an e2e script that expected auto-start.

200. assistant (2026-07-05T08:15:04.196Z)

Aha, there is an e2e CUI for the store-into-medium flow that still assumed auto-upload. I'm updating that as part of the fix so CI follows the new explicit-start path too.

201. assistant (2026-07-05T08:15:20.811Z)

The store-medium CUI now clicks the explicit start button. I'm running the full e2e CI path next, because that's the workflow most likely to catch any mismatch between unit tests and the browser app.

202. assistant (2026-07-05T08:16:35.118Z)

The e2e CI path passed too, including the store-medium CUI with the new explicit start step. I'm doing the final diff/branch safety checks now before committing and pushing this update to PR #158.

203. assistant (2026-07-05T08:16:57.513Z)

Everything still checks out after the e2e patch. I'm fetching and re-verifying the branch base now, then I'll stage only the touched files explicitly and commit.

204. assistant (2026-07-05T08:17:05.047Z)

Branch check is clean: I'm on ``fix/157-ingest-ux-polish``, and the branch still contains only the PR's own prior commit over ``origin/master``. Staging explicit paths now.

205. assistant (2026-07-05T08:17:16.635Z)

The staged set matches the intended files only. I'll do one more staged-stat check, then commit.

206. assistant (2026-07-05T08:17:22.160Z)

Staged diff is exactly the ingest UI/tests/e2e/locales set. Committing now.

207. assistant (2026-07-05T08:17:34.473Z)

Commit landed locally. I'm checking the branch ancestry one more time before pushing, to make sure only the two PR commits are ahead of `origin/master`.

208. assistant (2026-07-05T08:17:38.842Z)

Only the intended two commits are ahead. Pushing the PR branch now.

209. assistant (2026-07-05T08:17:48.739Z)

The PR branch is pushed. I'm cleaning up the temporary local mock/dev servers, then I'll update the PR with the new visual evidence and a short note.

210. assistant (2026-07-05T08:17:54.560Z)

Mock API is stopped. I'm identifying the Vite dev-server process by command line so I don't disturb unrelated Node processes.

211. assistant (2026-07-05T08:18:01.253Z)

Those are the Vite server process tree for port 5174. Stopping them now.

212. assistant (2026-07-05T08:18:20.054Z)

Temporary servers are cleaned up. Next I'm checking the existing screenshot-hosting branch/comment pattern so I can add the new after screenshots consistently instead of putting binaries into the PR branch.

213. assistant (2026-07-05T08:18:26.745Z)

PR #158 is still draft, and the previous screenshot comment uses the `assets-pr-158-screenshots` branch. I'm going to add the three new rendered-state images there and then post a short PR reply with the updated evidence.

214. assistant (2026-07-05T08:18:37.072Z)

There's already a dedicated screenshots worktree for the asset branch, so I'll reuse that instead of switching the PR worktree. Pulling it first, per the repo-freshness rule.

215. assistant (2026-07-05T08:18:47.304Z)

The asset worktree is clean and current. I'm copying in the new rendered-state screenshots now.

216. assistant (2026-07-05T08:19:03.840Z)

The new screenshot files are in the asset worktree. I'll commit and push them on the screenshot branch, again staging only those image paths.

217. assistant (2026-07-05T08:19:08.686Z)

Asset branch is current and the only untracked files are the three new screenshots. Staging and committing those now.

218. assistant (2026-07-05T08:19:20.287Z)

Screenshot commit is ready; I'm pushing the asset branch so the PR comment can render the images.

219. assistant (2026-07-05T08:19:38.735Z)

The screenshot branch is pushed. I'm posting a PR note with the new `selected` and `uploading` shots, plus the concrete no-auto-upload verification.

220. assistant (2026-07-05T08:20:02.476Z)

PR comment is posted with the new screenshots. I'm checking the remote PR/check status now, then I'll refresh the session handoff so the next window starts from the real pushed state.

221. assistant (2026-07-05T08:20:15.424Z)

Remote PR checks are all green and the branch is merge-clean; it is still marked draft. I'm locating the handoff files now so I can record the pushed commit, CI state, and PR comment link.

222. assistant (2026-07-05T08:20:27.684Z)

The active project memory appears to be under ``.claude/projects/C--Users-avidu-Projects-khelsutra/memory``. I'm reading the current handoff before editing it so I only update the real changed state.

223. assistant (2026-07-05T08:20:40.892Z)

That memory file is stale from the localization chapter, so I'm checking for the newer Alpha launch tracker handoff/current-state before overwriting anything.

224. assistant (2026-07-05T08:20:54.446Z)

There is a separate ``khelsutra-guru`` memory directory, which is likely the current Alpha-launch context. I'm reading that one now before updating.

225. assistant (2026-07-05T08:22:01.355Z)

I've got the thread state back in view. I'm going to do the last bit of housekeeping now: refresh the handoff with the explicit-start change, re-check the two worktrees/PR state, then I'll give you a concise status plus the prioritized remaining items.

226. assistant (2026-07-05T08:22:10.170Z)

The active memory directory is the same one the earlier handoff pointed at. I'm reading the handoff/current-state now so the update lands exactly where the next session will look first.

227. assistant (2026-07-05T08:22:18.986Z)

The handoff is a little stale now: it describes the first #158 push, before the explicit `?Start upload?` change and the new screenshots. I'm going to patch the memory docs to make that the next-session truth.

228. assistant (2026-07-05T08:22:44.549Z)

The handoff now captures the second PR update: no auto-submit, prominent filename title, and the new screenshot comment. I'm patching the memory index/current-state next so the start-here line matches reality.

229. assistant (2026-07-05T08:23:09.413Z)

Current-state is updated as well, including the no-network-before-start verification. I'm doing the memory index line now, then I'll verify the live PR/checks rather than relying on the earlier snapshot.

230. assistant (2026-07-05T08:23:25.052Z)

Memory is fresh. I'm switching back to repo verification now: fast-forward pulls first, then clean-status/PR/check state, because the branch and CI can move while we're talking.

231. assistant (2026-07-05T08:23:34.499Z)

The PR branch is clean and exactly two commits over ``origin/master``: the original UX polish and

the explicit-start follow-up. I'm checking the screenshot asset branch and live PR metadata next.

232. assistant (2026-07-05T08:23:59.387Z)

The live PR state matches the update: PR #158 is still open as a draft, merge state clean, and all four remote checks are green on `dc9dc23`. I'm doing one tracker/doc read now so the pending list reflects the current Alpha tracker rather than just memory.

233. assistant (2026-07-05T08:24:15.224Z)

Engine tracker checkout is up to date. I'm pulling just the relevant tracker rows and the confirmed live-bugs note now, then I'll wrap this up cleanly.

234. assistant (2026-07-05T08:24:33.477Z)

One useful correction surfaced from the tracker memory: #479/#480 were config-fixed on CP rev `00011-dx6`; the remaining risk is a full image rebuild/deploy for the newer SPA/copy/UX plus the medium-picker trap. I'm correcting my handoff wording so it doesn't resurrect fixed bugs as pending.

235. assistant (2026-07-05T08:24:45.868Z)

That correction is in the handoff. I'm updating the memory index's bug line too, since it was still shouting ?live? even though the config fix landed.

236. assistant (2026-07-05T08:25:00.966Z)

The index now says the same thing as the detailed bug note: config fixed, rebuild/UX still pending. I'm doing one last quick readback of the top memory lines and then I'll give you the handoff.

237. assistant (2026-07-05T08:25:07.046Z)

One more stale line popped out in the handoff readback: it still named CP rev `00010`. I'm correcting that to the current config-fixed rev so the deploy notes don't mislead the next pass.

238. assistant (2026-07-05T08:25:30.017Z)

Done. I incorporated the screenshot feedback into PR [#158](https://github.com/avidullu/khelsutra/pull/158):

- File selection no longer auto-starts upload/indexing.
- The selected video filename now appears near the top of the card in a larger title style.
- Upload starts only from the explicit `Start upload and find rallies` button.
- I posted the updated screenshot/comment here: [PR comment](https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885367767).

Verification is solid: local web/site tests, typecheck, coverage, e2e CI, Playwright visual checks, and remote GitHub checks are all green on `dc9dc23`. PR #158 is still a draft; I did not mark it ready.

I also refreshed the handoff/memory files, including [session-handoff.md](C:/Users/avidu/.claude/projects/C--Users-avidu-Projects-khelsutra-guru/memory/session-handoff.md).

Prioritized pending items:

1. Decide whether to mark PR #158 ready for review, then review/merge it.
2. Rebuild/deploy the full image so #154 copy plus #157/#158 SPA UX go live, preserving CP rev `00011-dx6` config fixes.
3. Run owner browser CUJ and record GO/NO-GO for A13.
4. Fix the remaining medium-picker UX trap: gs:// source link vs ?Cloud bucket? destination.
5. Wider-invite gates: A10/A11, least-privilege service account, retention/processor/acceptable-

use wording, kn/hi native-speaker review.

6. Normal backlog: #329 async compile, #471 durable state, `gen\_service\_yaml.py --revision-suffix`, rally finding-B promotion.

239. user (2026-07-05T08:49:38.692Z)

Can you review [Khelsutra/badminton-highlight-indexer#481](https://github.com/Khelsutra/badminton-highlight-indexer/pull/481) while your PR is being reviewed?

240. assistant (2026-07-05T08:49:53.615Z)

I'll treat this as a real PR review, not a skim: pull the engine repo first, inspect the PR diff and discussion, check out the PR in an isolated worktree, run the expected gates, then either post findings or a verified LGTM if it earns one.

241. assistant (2026-07-05T08:49:59.999Z)

I'm using the code-review and GitHub workflows here: first to keep the review focused on correctness/security/perf, and second to pull PR metadata/comments/checks before deciding whether to post LGTM or findings.

242. assistant (2026-07-05T08:50:17.738Z)

The GitHub connector timed out while starting, so I'm falling back to `gh` plus an isolated worktree. The repo itself is already fast-forwarded, so we're reading current remote state.

243. assistant (2026-07-05T08:50:28.030Z)

PR #481 is open, not draft, CI-green, and cleanly mergeable. It's a server-side durable-job/dedup change, so I'm going to review the DB scoping/idempotency behavior carefully before even thinking about LGTM.

244. assistant (2026-07-05T08:50:33.820Z)

The main checkout is actually sitting on the PR branch already, so I'm not going to disturb it. I fetched the PR head as `origin/pr/481` and will use a separate detached review worktree for test runs and file reads.

245. assistant (2026-07-05T08:50:49.220Z)

The isolated worktree is ready at `C:\Users\avidu\Projects\khelsutra-guru\wt-pr481-review` on PR head `84403e7`. I'm reading the changed implementation plus nearby route/auth/job code now.

246. assistant (2026-07-05T08:51:02.617Z)

The new surface is small, but it touches tenant scoping and ?don't double-spend GPU? semantics, so I'm narrowing into three places: `process\_video`, the new `/api/jobs` router, and the job-store query semantics.

247. assistant (2026-07-05T08:51:27.562Z)

I found the key tests the PR adds: dedup/no-second-row, terminal-job opt-out, active-only listing, tenant scoping, and compile-job exclusion. Next I'm checking the edge-auth tenant resolver and job schema/migrations, because subtle owner-id behavior is where this kind of PR usually gets sharp edges.

248. assistant (2026-07-05T08:51:48.020Z)

One potential correctness issue is emerging: the dedup check is a read-before-insert, not an atomic insert-or-return. I'm going to try to reproduce the concurrent double-submit case locally before treating it as a finding.

249. assistant (2026-07-05T08:52:28.281Z)

The race reproduced deterministically: two concurrent identical ``/api/process`` requests both got different ``job_id``s and two queued rows before either could see the other. I'm going to finish the surrounding review and run the relevant gates, but this is likely a request-changes finding because it undercuts the PR's "no second paid GPU run" guarantee.

250. assistant (2026-07-05T08:52:38.921Z)

Remote CI is green, so the current tests don't catch this race. I'm running the PR's focused `async-job` tests and then the local review gates I can run cheaply from this worktree.

251. assistant (2026-07-05T08:53:50.278Z)

Focused tests, targeted ruff, and targeted mypy are clean. I've got a blocker already, but I'm still running the repo's full offline suite once so the PR comment can separate "tests pass" from "behavioral race remains."

252. assistant (2026-07-05T08:56:00.292Z)

Full offline suite passed locally: ``1663 passed, 10 skipped``, coverage floor 70 satisfied. I'm running the full lint/type/diff hygiene checks now, then I'll post the review.

253. assistant (2026-07-05T08:56:21.175Z)

I'm going to post a blocking review comment rather than LGTM. The code is otherwise clean and the gates pass, but this one race is exactly on the PR's paid-GPU protection path.

254. assistant (2026-07-05T08:56:43.010Z)

The review comment is posted: ``#issuecomment-4885456620``. I'm refreshing the handoff with the PR #481 review status now, so the next pass knows we're waiting on an atomic-dedup fix rather than an LGTM.

255. assistant (2026-07-05T08:57:09.143Z)

A sibling/current memory update landed before mine and already has a newer browser-CUJ block. I'm going to patch only the stale #481 sentence there, preserving the new CUJ details.

256. assistant (2026-07-05T08:57:42.093Z)

The session handoff had the same stale "481 awaiting LGTM" wording, so I'm correcting that single line too.

257. assistant (2026-07-05T08:58:11.771Z)

Handoff now says #481 is not LGTM and names the atomic-dedup blocker. I'm checking the index/worktree state once more, then I'll summarize.

258. assistant (2026-07-05T08:58:18.929Z)

The memory index still had a small stale phrase saying #481 was merely CI-green. I'm making that line precise too: CI-green but review-blocked.

259. assistant (2026-07-05T08:58:37.577Z)

I reviewed [PR #481](https://github.com/Khelsutra/badminton-highlight-indexer/pull/481) and did **not** LGTM it.

Finding posted: [comment #4885456620](https://github.com/Khelsutra/badminton-highlight-indexer/pull/481#issuecomment-4885456620)

Blocking issue: ``/api/process`` dedup is read-before-insert, so two concurrent identical submits

can both miss the active job and insert two queued rows. I reproduced it locally with two concurrent `TestClient` posts: both got different `job\_id`s for the same `video\_path`.

Verified clean otherwise on head `84403e7`:

- `tests/test\_async\_jobs.py`: 24 passed
- Full suite: 1663 passed, 10 skipped, coverage floor satisfied
- `ruff check .`: clean
- `mypy backend training`: clean
- `git diff --check origin/master...HEAD`: clean
- Remote CI: green

I refreshed the handoff/memory to show #481 is CI-green but review-blocked on atomic dedup.

260. user (2026-07-05T08:58:38.196Z)

can you quickly resolve the only comment in your #158 and then respond

261. assistant (2026-07-05T08:58:48.133Z)

I'll pick up #158 directly: pull the PR worktree first, inspect the current PR comment/thread, make the smallest fix that answers it, run the relevant checks, push, and reply on GitHub.

262. assistant (2026-07-05T08:58:57.732Z)

I'm using the PR-comment handler workflow because this is exactly ?address the remaining PR feedback and respond.? I'll skip a big plan unless the comment turns out broader than it sounds.

263. assistant (2026-07-05T08:59:15.245Z)

The actionable comment is the medium-picker trap: a pasted `gs://` source link should not silently behave like a storage destination just because ?Cloud bucket? is selected. I'm going to make the cloud-link path submit as an existing video source regardless of the storage-medium picker; the picker will remain for file uploads.

264. assistant (2026-07-05T08:59:19.804Z)

The current code literally hides the ?cloud link? form in non-local medium mode, which explains the live trap. I'm changing that so the link form remains a source form in every medium mode, and only the file-upload path is gated by the storage-destination field.

265. assistant (2026-07-05T08:59:39.572Z)

The fix is small but important: split ?file upload is blocked until destination exists? from ?cloud link source can submit now.? I'm patching the component and the medium-picker tests around that behavior.

266. assistant (2026-07-05T09:00:02.135Z)

The component now keeps the cloud-link source form visible in bucket/Drive mode and leaves it enabled even when the upload destination is empty. I'm adding the regression test that would have caught the live trap.

267. assistant (2026-07-05T09:00:17.833Z)

The regression test asserts exactly the stuck case: bucket mode selected, destination empty, pasted `gs://` source still posts to `/api/process` and never calls `/api/assets`. I'm running the focused test file now.

268. assistant (2026-07-05T09:00:32.547Z)

The implementation works, but my new test expected `onLoaded` before the one-second poll tick. I'm tightening the test to advance that poll tick instead of making the component behavior less

realistic.

269. assistant (2026-07-05T09:00:43.021Z)

The regression now follows the actual submit?poll cadence. I?m rerunning the focused tests, then I?ll do typecheck/full relevant gates before pushing.

270. assistant (2026-07-05T09:01:01.297Z)

The regression doesn?t need to wait for the completed job; the useful assertion is that the source link posts directly to `/api/process` and skips `/api/assets`. I?m trimming the test back to that faster contract.

271. assistant (2026-07-05T09:01:14.386Z)

The test is now scoped to the trap itself. I?m rerunning the focused tests.

272. assistant (2026-07-05T09:01:31.485Z)

The focused regression is green now. I?m running the companion upload tests and typecheck, then I?ll commit/push the small fix and reply on the PR.

273. assistant (2026-07-05T09:02:45.108Z)

The patch does what the reviewer asked: cloud/host links stay visible and submit as a source even when bucket/Drive storage is selected, while local file uploads still require a destination. I?m rerunning the focused web checks now before committing.

274. assistant (2026-07-05T09:02:57.742Z)

Focused tests and typecheck are green. I?m doing the required branch/HEAD sanity check now, then I?ll commit only the two touched files and push.

275. assistant (2026-07-05T09:03:07.608Z)

Committed as `c94d485`. I?m doing the pre-push range check so the branch contains only the #158 commits, then I?ll push and answer the PR comment.

276. assistant (2026-07-05T09:03:18.300Z)

Push is up. Now I?m posting the PR response with exactly what changed and what I verified, so the reviewer has a clear thread to close.

277. assistant (2026-07-05T09:03:31.959Z)

The PR reply is posted. I?m doing the small handoff cleanup now so the next window won?t chase the already-fixed comment.

278. assistant (2026-07-05T09:03:42.251Z)

I?m updating the handoff to reflect that the medium-picker trap is no longer an open #158 code-review item; it?s now a merge/deploy item with the fix pushed.

279. user (2026-07-05T09:04:24.246Z)

pls do merge the PR now

280. assistant (2026-07-05T09:04:39.745Z)

Got it. I?ll treat this as merge #158 now, but first I?m checking the PR state and CI so we don?t merge a draft or a red head by accident.

281. assistant (2026-07-05T09:04:49.752Z)

#158 is still draft and the `web` CI job is still running, so I can't merge it quite this second. I'm marking it ready for review now, then I'll merge as soon as the last required check finishes green.

282. assistant (2026-07-05T09:04:55.310Z)

PR is out of draft now. I'm watching the remaining GitHub check; once it's green I'll squash-merge and clean up the branch.

283. assistant (2026-07-05T09:05:38.506Z)

All required checks are green now. I'm merging #158 with the usual squash merge and deleting the PR branch.

284. assistant (2026-07-05T09:06:04.558Z)

#158 is merged now; the squash landed on `master` as `0e45e8f`. I'm confirming the PR/issue state and refreshing the handoff so future sessions see the merged state, not the old draft.

285. assistant (2026-07-05T09:06:15.409Z)

I'm trimming the stale ?#158 is still open? bits from memory now. Tiny bookkeeping, but it prevents the next session from chasing a ghost task.

286. assistant (2026-07-05T09:07:02.729Z)

Session handoff is current. I'm making the matching top checkpoint update in `current-state.md`, then I'll give you the crisp merge result.

287. assistant (2026-07-05T09:07:59.623Z)

Merged [khelsutra#158](https://github.com/avidullu/khelsutra/pull/158) now.

It was still draft, so I marked it ready, waited for the last `web` check to pass, then squash-merged it to `master` as `0e45e8f`. [Issue #157](https://github.com/avidullu/khelsutra/issues/157) auto-closed. I also refreshed the session handoff/current-state so the next session sees #158 as merged, not pending.

Checks green before merge: `site`, `web`, `layout`, and Cloudflare Worker build.

288. user (2026-07-05T09:09:57.826Z)

while the CI runs can you do a logic review of #481 and LGTM based on that

289. assistant (2026-07-05T09:10:15.089Z)

I'll do a logic pass on #481 now. One boundary: I won't post an unconditional LGTM while CI is still running, because your standing review gate requires verified green checks; if the logic is sound, I'll post a ?logic looks good, pending CI/local gates? style note or wait and stamp it once the checks are actually green.

290. assistant (2026-07-05T09:10:28.305Z)

The author pushed `d27f2d7`, which directly targets my prior atomic-dedup finding. CI still has a few long jobs in progress, so I'm checking the actual diff and the concurrency path locally now.

291. assistant (2026-07-05T09:10:43.245Z)

I see the local checkout is already on `feat/jobs-queue-and-dedup` at `d27f2d7` and clean, which is convenient for the review. I'm reading the DB helper and route integration around the new atomic claim, then I'll run at least the concurrency regression locally while the slower gates continue.

292. assistant (2026-07-05T09:10:57.306Z)

The key race fix is in the right place: ``/api/process`` no longer does a pre-read followed by ``create_job``; it routes through one DB helper before dispatch, and dispatch only happens for the caller that actually created the row. I'm checking schema/default assumptions and the failure paths next, because that's where this kind of fix usually hides sharp edges.

293. assistant (2026-07-05T09:11:08.130Z)

Schema defaults line up: ``kind`` is nullable so the existing ``(kind IS NULL OR kind != 'compile')`` process-job filter preserves old rows, and the new helper writes the same columns as ``create_job``. I'm running the targeted async-job tests now and also checking whether the PR has any new edge-auth or tenant-scope holes.

294. assistant (2026-07-05T09:11:26.200Z)

The targeted regression suite is green locally, including the new 12-thread atomic claim test. CI has all quick jobs green and only the full offline suite still pending, so I'm starting the local full gate now rather than relying on the PR comment's claimed numbers.

295. assistant (2026-07-05T09:13:59.476Z)

Local full suite, lint, mypy, and diff-check are all green. I'm doing one last CI poll; if the GitHub full suite has turned green too, I can post the verified LGTM.

296. assistant (2026-07-05T09:14:15.001Z)

CI just turned fully green too, and the merge state is clean. The previous blocker is resolved on both logic and verification, so I'm posting the LGTM with the exact local and CI gates I checked.

297. assistant (2026-07-05T09:14:30.971Z)

LGTM is posted. I'm doing a quick memory update now because #481 changed state from ?blocked on atomic dedup? to ?logic reviewed + LGTM, ready for merge.?

298. assistant (2026-07-05T09:14:37.599Z)

The handoff now still says #481 is blocked, so I'm correcting just those stale lines. Then I'll stop; no branch changes to commit here.

299. assistant (2026-07-05T09:15:10.504Z)

Session handoff is corrected. I'm applying the same state correction to the live checkpoint.

300. assistant (2026-07-05T09:15:42.305Z)

Done. I posted LGTM on [#481](https://github.com/Khelsutra/badminton-highlight-indexer/pull/481#issuecomment-4885506095).

I verified the atomic-dedup logic on ``d27f2d7`` and ran the full gate locally:

```
`python run_all_tests.py --cov-fail-under=70` ? 1671 passed, 3 skipped
`ruff check .` ? clean
`mypy backend training` ? clean
`git diff --check origin/master...HEAD` ? clean
```

GitHub CI is also fully green now. I refreshed the handoff/current-state too.

301. user (2026-07-05T09:21:02.092Z)

what is the : A10/A11 gating?

302. assistant (2026-07-05T09:21:12.520Z)

I'll pull the tracker repo first, then read the launch readiness rows for A10/A11 so I can answer from the actual source instead of half-remembering the shorthand.

303. assistant (2026-07-05T09:21:22.481Z)

Repo is up to date and clean on `master`. I'm opening the readiness tracker and the launch review sections now.

304. assistant (2026-07-05T09:21:40.924Z)

A10/A11 are **owner/legal gates** around what we are allowed to claim, not **write more code?** gates.

- **A10:** Decide the alpha **ship-gate target**: which metric, which corpus slice, minimum threshold, significance rule, and exactly what quality/model claims are allowed. Until this is recorded, we should not claim **the owned model is ready?** or make broad detection-quality promises.

- **A11:** Resolve or explicitly defer **WASB C3 provenance**: whether the WASB checkpoint/weights provenance is commercially clean enough for model sharing/commercial model claims. If deferred, we need explicit copy like **no commercial model sharing?** and keep alpha messaging scoped to invite-only video processing, not model launch.

Practical effect:

- For the **current tiny 4-email alpha**, A13 said conditional GO is still possible if copy is honest and owner browser-CUJ passes.
- For **wider invites**, it is NO-GO until A10/A11 are closed or explicitly deferred with guardrails, plus least-privilege SA and retention cleanup.

Source: [ALPHA_LAUNCH_READINESS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ALPHA_LAUNCH_READINESS.md:109) and [ALPHA_LAUNCH_REVIEW_2026-07-04.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md:18).

305. user (2026-07-05T09:23:14.603Z)

Help me plan for these A10/A11 then, the CUJ found some issues which are being worked upon and I want to clear this headwinds

306. assistant (2026-07-05T09:23:30.000Z)

I'll map A10/A11 into a small decision package: what you need to decide, what evidence already exists, what conservative default keeps the alpha moving, and what doc/PR artifact would close the tracker rows. I'm pulling the provenance/ship-gate references now so the plan is grounded in the actual repo language.

307. assistant (2026-07-05T09:24:24.685Z)

Yes. I'd clear A10/A11 with a **decision record**, not by trying to finish all model/legal uncertainty right now.

My Recommendation

For the current alpha, separate the decisions like this:

1. **A10: Close for product alpha, defer model-quality ship gate.**
 - Decision: this alpha is a **product/CUJ alpha**, not a model-quality or owned-model launch.
 - Allowed claim: **KhelSutra helps invited testers process badminton videos into editable highlight reels?**
 - Disallowed claims: **commercially ready detector?** **owned model is production-ready?** **accuracy guarantees,** or **quality benchmarks in marketing copy.**
 - Keep issue [#172](https://github.com/Khelsutra/badminton-highlight-indexer/issues/172) open

for the real model ship-gate calibration.

2. **A11: Explicitly defer WASB C3 with guardrails.**

- Decision: WASB checkpoint provenance remains unresolved.
- Current 4-email alpha may proceed only as a private, owner-operated, non-public testing loop.
- No weight/model sharing, no commercial model claims, no public/wider invite wave until C3 is cleared or a different cleared detector path is chosen.
- This is not legal advice; I'd treat wider/public/commercial use as counsel-gated.

Why this works

The CUJ issues are engineering reliability issues. A10/A11 are messaging and governance issues. You can clear the governance headwind now by saying: "We are not launching a model; we are testing an invite-only product workflow."

Decision Record Shape

I'd create/update a short doc with:

- **Alpha scope:** current 4-email allowlist only.
- **A10 decision:** no quality/model claims for this alpha; quality evidence remains internal.
- **Allowed claims:** editable highlight reel workflow, invite-only alpha, experimental automatic rally finding, review-before-sharing.
- **Blocked claims:** production accuracy, owned/commercial model, WASB-cleared model, public availability.
- **A11 decision:** WASB C3 deferred with no model/weight sharing and no wider invites.
- **Exit criteria for wider invite:** CUJ fixes green, least-privilege SA, retention/deletion cleanup, A10 model gate ratified or claims remain strictly non-model, A11 counsel/provenance resolved or replacement path chosen.

Concrete PR

One small docs PR can close this:

- Update `docs/ALPHA\_LAUNCH\_READINESS.md` rows A10/A11.
- Add a "Launch Decision: current alpha claims and WASB-C3 guardrail" section.
- Update A13 review status to say A10/A11 are closed for "current limited alpha", not for wider/public launch.
- Leave #172 open as the model ship-gate calibration follow-up.

My suggested status after that PR:

- **A10:** DONE for current product alpha; model-quality ship gate deferred to #172.
- **A11:** DONE as explicit defer-with-guardrails for current 4-email alpha; wider/public remains blocked.

308. user (2026-07-05T09:28:26.888Z)

sounds good and for the alpha testers, we may have <25 for sure

309. assistant (2026-07-05T09:28:44.758Z)

That helps. I'd phrase the decision as:

A10/A11 Scope

Current alpha is capped at "fewer than 25 individually allowlisted testers". It is an invite-only product workflow alpha, not a public launch and not a model/commercial-quality launch.

Guardrails

- No public signup, no open beta language.
- Each tester is explicitly allowlisted by email.
- Claims stay workflow-level: upload video, auto-find candidate rallies, edit/pick, compile reel.
- Copy must say rally finding is experimental and should be reviewed before sharing.
- No owned-model, commercial detector, accuracy, or "production-ready" claims.
- WASB C3 remains deferred; no model/weights sharing.

For A10, I'd close it as: ****DONE** for limited product alpha; model ship-gate remains deferred to #172.**

For A11: ****DONE** for limited product alpha via explicit defer-with-guardrails; public/wider/commercial launch remains blocked until provenance/counsel path is resolved.**

The <25 cap is a nice clean line: small enough to be controlled, large enough to test real UX. It should still sit behind the CUJ fixes, honest copy, auth, retention wording, and least-privilege cleanup before invites go out.

310. user (2026-07-05T09:28:46.591Z)

can you make the PR, ensure all the docs/links github issues are updated

311. assistant (2026-07-05T09:29:06.104Z)

I'll do this as a small docs PR in the engine repo, with the tracker updated and issue comments/labels aligned. I'll use the PR-publishing workflow and keep the branch isolated from `origin/master` so it doesn't accidentally ride on any sibling work.

312. assistant (2026-07-05T09:29:32.908Z)

The repo is clean on `master`. I've got the publish/doc workflow loaded; next I'm checking `gh`, then I'll cut `codex/a10-a11-alpha-guardrails` from `origin/master` and keep this PR to docs only.

313. assistant (2026-07-05T09:29:38.588Z)

Nice, #481 has already landed on `origin/master` as `0a7b3ff`, so the docs PR can reference that as baked into the next deploy path. I'm branching from that exact remote head now.

314. assistant (2026-07-05T09:29:58.316Z)

I'm going to update the tracker, the A13 review summary/scorecard, and a short decision section so the allowed claims are explicit and linked. I'm also checking whether there's an existing WASB-C3 GitHub issue to reference; if not, I'll file or comment appropriately instead of leaving the guardrail as folklore.

315. assistant (2026-07-05T09:30:13.207Z)

There's an existing ship-gate issue (#172), but no dedicated WASB-C3 provenance issue. I'm creating that tracker now so A11 has a concrete GitHub anchor rather than being only a doc phrase.

316. assistant (2026-07-05T09:30:43.335Z)

I'm editing three docs: the tracker, the A13 review note, and the docs index. The core move is: A10/A11 are closed for a capped <25 product alpha with strict claim guardrails, while #172 and new #486 remain the follow-up gates for model/public/commercial posture.

317. assistant (2026-07-05T09:31:25.472Z)

The tracker now records the <25 allowlist and separates product-alpha claims from model/public claims. I'm updating the A13 review so its scorecard and recommendations don't contradict the new A10/A11 decision.

318. assistant (2026-07-05T09:33:22.411Z)

I'll finish this as a documentation PR from the fresh indexer `origin/master`, then wire the GitHub issue trail so the A10/A11 decision is easy to audit later. I've got the branch and core docs edits in place; I'm tightening the index entry and then I'll run the doc checks before pushing.

319. assistant (2026-07-05T09:33:38.792Z)

A quick diff scan turned up a few older phrases in the review doc that still said A10/A11 were ?owner-gated? in the original report body. I?m going to clean those up too so the addendum is internally consistent, not just patched at the top.

320. assistant (2026-07-05T09:33:50.934Z)

I?m also avoiding brittle line-number references where the new decision record shifted sections around. The docs will point to sections/issues rather than stale `:109-110` style anchors where that would age badly.

321. assistant (2026-07-05T09:34:17.809Z)

The README index is now aligned, and I cleaned the obvious stale A10/A11 language in the review. Next I?m checking for leftover contradictions and then I?ll run the markdown/link checks.

322. assistant (2026-07-05T09:34:30.712Z)

The consistency scan is useful: a few remaining ?4-email? mentions are historical/current-state facts, but I?m converting the launch-policy ones to the new `<25` cap and dropping stale readiness line references. That should leave the docs crisp instead of half-old, half-new.

323. assistant (2026-07-05T09:35:27.922Z)

The remaining old anchors are now mostly factual evidence references or historical changelog language. I?ll run one more targeted scan, then move to checks.

324. assistant (2026-07-05T09:35:47.227Z)

Docs wording is now internally consistent around the new cap. I?m going into verification mode: markdown links, whitespace, and a final diff pass before I commit.

325. assistant (2026-07-05T09:35:57.062Z)

Whitespace check is clean. The link-check command exists, and I spotted one last old ?4 trusted testers? sentence in the SA section, so I?m verifying that patch and then running the actual checker.

326. assistant (2026-07-05T09:36:05.232Z)

Markdown link checking is clean. I?m doing the final review diff now, then I?ll commit with only these three doc files staged.

327. assistant (2026-07-05T09:36:16.202Z)

Verification passed for links and whitespace. I?m doing the branch-safety checks now, then I?ll stage only the three touched docs and make the commit.

328. assistant (2026-07-05T09:36:19.572Z)

Branch base is clean: it?s directly on `origin/master` at #481?s merge commit, with no extra commits yet. I?m committing the docs-only change now.

329. assistant (2026-07-05T09:36:27.917Z)

Commit is in. I?m re-verifying the branch contains only this commit before pushing and opening the draft PR.

330. assistant (2026-07-05T09:36:31.125Z)

The branch contains exactly one commit, the docs-only guardrail commit. Pushing now, then I?ll create the PR and post the issue comments.

331. assistant (2026-07-05T09:36:41.465Z)

Push succeeded. I'm opening the PR as draft, since this is my branch and there's no explicit ready-for-merge instruction yet. The body will call out the checks and keep #172/#486 open as gates.

332. assistant (2026-07-05T09:36:57.061Z)

PR is up: #488. I'm adding the cross-reference comments to #172 and #486 now so the tracker issues point back to the exact docs PR.

333. assistant (2026-07-05T09:37:13.014Z)

One more polish pass: now that #488 exists, I'm going to add the PR number into the tracker changelog/rows so future readers don't have to hunt through GitHub history. Then I'll amend and force-push the same branch.

334. assistant (2026-07-05T09:37:33.258Z)

The PR cross-link is now in the docs. Re-running the doc checks before amending.

335. assistant (2026-07-05T09:37:41.443Z)

Checks are still clean. I'm amending the existing commit with just those PR cross-links and force-pushing with lease.

336. assistant (2026-07-05T09:38:03.834Z)

The amended commit is pushed. Given you called out draft state on the last PR, I'm checking #488's state now and I'll make it ready for review if it's still draft.

337. assistant (2026-07-05T09:38:10.453Z)

#488 is still draft and CI has queued. I'm marking it ready for review now so it doesn't get stuck in the same draft limbo.

338. assistant (2026-07-05T09:38:25.509Z)

The PR is ready for review now. I'm refreshing the session handoff next, because this is exactly the kind of checkpoint we'll want future windows to pick up cleanly.

339. assistant (2026-07-05T09:38:40.048Z)

The memory handoff still says A10/A11 are owner actions, so I'm patching the top checkpoint to reflect #488 and the two issue comments. I'll keep the CP rebuild spec as the main next engineering path, but remove the stale governance blocker.

340. assistant (2026-07-05T09:39:24.115Z)

Handoff is patched. I'm adding the same micro-checkpoint to `current-state` so the dashboard view has the new #488 status up top.

341. assistant (2026-07-05T09:39:52.501Z)

Memory is refreshed. I'm doing one last status/CI snapshot so I can give you an accurate close-out.

342. assistant (2026-07-05T09:40:00.122Z)

#488 is ready and mergeable; 10 of 11 CI jobs are green, with only the full offline suite still running. I'll give that one a little time so I can close with a cleaner status.

343. assistant (2026-07-05T09:41:13.691Z)

Nice, CI went fully green on #488. I'm updating the handoff checkpoint from ?queued? to ?green,? then I'll wrap this up.

344. assistant (2026-07-05T09:42:04.440Z)

All right, memory now reflects the green PR. I'm taking the final repo/PR snapshot and then I'll summarize the work.

345. assistant (2026-07-05T09:42:23.474Z)

Done. Opened [engine PR #488](https://github.com/Khelsutra/badminton-highlight-indexer/pull/488) and marked it ****ready for review****. It is ****not draft****, is ****mergeable****, and all GitHub CI checks are green.

What changed:

- Updated `ALPHA\_LAUNCH\_READINESS.md`, `ALPHA\_LAUNCH\_REVIEW\_2026-07-04.md`, and `docs/README.md`.
- Recorded A10/A11 as closed only for the limited product alpha: `<25` individually allowlisted testers, invite-only, workflow-only claims.
- Kept model/public/commercial gates open via [#172](https://github.com/Khelsutra/badminton-highlight-indexer/issues/172) and new [#486](https://github.com/Khelsutra/badminton-highlight-indexer/issues/486).
- Added comments linking #172 and #486 back to #488.
- Refreshed the session handoff/current-state memory.

Verified:

- Local: `python -m tools.check\_md\_links`, `git diff --check`
- GitHub CI: full green, including ruff, mypy, markdown links, leak scan, full suite, wheel smoke, cross-OS fixtures.

346. user (2026-07-05T09:42:24.157Z)

Can you take over the review of [Khelsutra/badminton-highlight-indexer#487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487) ?

347. assistant (2026-07-05T09:42:41.674Z)

I'll take it over as a real review, not a skim. I'm going to sync the repo first per your freshness rule, inspect the PR diff and its CI/review threads, then run the relevant local gates before deciding whether to post LGTM or findings.

348. assistant (2026-07-05T09:43:00.555Z)

Using the code-review workflow here, with GitHub triage for PR metadata and local git/gh for the diff and CI. Since #487 is the CP rebuild spec, I'm checking both markdown mechanics and the logic: whether the plan actually covers the failure modes from the CUJ without sneaking in brittle deploy assumptions.

349. assistant (2026-07-05T09:43:19.359Z)

#487 itself is clean on GitHub: no prior review threads, two docs files, CI all green, mergeable right now. I'm putting the PR head into a separate review worktree so I can inspect line numbers and run the local gates without disturbing the #488 branch.

350. assistant (2026-07-05T09:43:40.791Z)

I've got the PR content now. One subtle area I'm digging into is whether the proposed durable key matches the engine's existing `video\_id` contract, because the whole restart/cache plan depends on that being exactly the same identifier, not merely ?some MD5-ish thing.?

351. assistant (2026-07-05T09:43:54.167Z)

The PR has an existing worktree at `wt-cp-rebuild-spec`, so I'm using that rather than creating

a duplicate. The first logic pass is mostly consistent; I'm checking code anchors now for ?verified? claims and looking for anything that would make the planned implementation impossible or misleading.

352. assistant (2026-07-05T09:44:08.990Z)

The markdown checks are clean. One code anchor note: master after #481 already has some ``video_id`-at-create` machinery, so I'm checking whether #487's narrative is stale there or whether it's specifically about remote worker re-association. If it's just imprecise wording, that's not necessarily a blocker; if it mis-scopes P3/P4, it is.

353. assistant (2026-07-05T09:44:37.369Z)

A concrete nuance is emerging around upload backend selection. The live cloud-serving config likely has ``input_backend=gcs``, but the repo deliberately supports ?cloud input_root with default input_backend? after #467, so I'm checking whether the spec should preserve that root-scheme behavior instead of narrowing P2 to ``storage.input_backend == "gcs"``.

354. assistant (2026-07-05T09:44:52.571Z)

The P2 backend-selection wording looks a little narrow, but the generator currently sets ``input_backend=gcs``, so it does not break the intended alpha path. I'm checking one deeper thing now: whether the cache-hit path can be implemented before job creation, because #481 moved process submission into an atomic ?claim or create? path.

355. assistant (2026-07-05T09:45:05.086Z)

The cache-hit plan is feasible, but it will force a route-level change: today ``/api/process`` deliberately avoids hashing before it creates the queued job, and the process request does not yet carry a submitted ``video_id``. I'm verifying whether the spec says that explicitly enough or whether this deserves a review note.

356. assistant (2026-07-05T09:45:17.617Z)

The route-level cache change is called out as P4, and I don't think it blocks the spec: it will need to move a cheap MD5/metadata lookup ahead of the #481 claim, but that's exactly the planned deliverable. I'm starting the local pre-LGTM gates now: docs links/diff already passed; next are ruff, mypy, and the full coverage suite.

357. assistant (2026-07-05T09:45:57.561Z)

Ruff, mypy, and leak scan are clean. I'm running the full offline suite with the repo's coverage floor now; that's the one gate that earns an actual LGTM.

358. assistant (2026-07-05T09:48:47.972Z)

Full local suite passed: 1664 passed, 10 skipped, coverage 85.49% over the 70% floor. I don't have any blocking findings from the spec review, so I'm posting an LGTM with the exact verification I reproduced.

359. assistant (2026-07-05T09:49:14.663Z)

LGTM is posted on #487. I'm refreshing the handoff one more time so the next session knows #487 has a verified review rather than treating it as unreviewed.

360. assistant (2026-07-05T09:49:20.692Z)

The PR comment is visible, and #487 is mergeable with green CI. I'm only patching memory now; no repo files are being touched.

361. assistant (2026-07-05T09:50:49.864Z)

Done. I took over review of [#487](https://github.com/Khelsutra/badminton-highlight-

indexer/pull/487) and posted an LGTM: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487#issuecomment-4885587746).

No blocking findings. I checked the spec logic against the current code anchors and reproduced the gates locally:

```
- `python -m tools.check_md_links`  
- `git diff --check origin/master...HEAD`  
- `ruff check .`  
- `mypy backend training`  
- `python -m tools.leak_scan`  
- full suite with coverage: `1664 passed, 10 skipped`, coverage `85.49%` vs floor `70%`
```

GitHub CI is also fully green and the PR is mergeable. I left one non-blocking sequencing note: if #488 merges first, #487 may need a tiny README/docs rebase.

362. user (2026-07-05T09:53:39.236Z)

lgtn

363. assistant (2026-07-05T09:53:59.383Z)

Got it. I'll treat that as the owner go-ahead for #487, verify it is still green/mergeable, then squash-merge and clean up the branch.

364. assistant (2026-07-05T09:54:09.878Z)

That worktree path seems to have disappeared from under me, so I'm switching to the main repo checkout and re-verifying from there. No merge has happened yet.

365. assistant (2026-07-05T09:54:26.366Z)

It looks like #487 has already been merged remotely by the time I rechecked, so there's nothing left to merge there. I'm checking #488 now because #487 touched the same README area and may have changed its mergeability.

366. assistant (2026-07-05T09:54:42.355Z)

#487 was merged by you at commit `cc5e83a`. As expected, #488 is now based behind the README change, so I'm going to rebase #488 onto the new `origin/master` and keep both docs index entries intact.

367. assistant (2026-07-05T09:54:47.960Z)

The #488 branch is clean and has only its one docs commit over the old base. Rebase next.

368. assistant (2026-07-05T09:54:56.273Z)

The expected README conflict happened. I'm resolving it by keeping #487's new CP rebuild row and #488's updated alpha-readiness row together.

369. assistant (2026-07-05T09:55:00.496Z)

I found the exact conflict: the alpha tracker row from #488 and the new CP rebuild row from #487. I'm replacing the conflict block with both rows in the right order.

370. assistant (2026-07-05T09:56:07.175Z)

Picking back up from the handoff: #487 is already merged on remote, and I'm finishing the cleanup that merge forced on our #488 docs branch. I'm going to continue the rebase, push the rebased PR, and re-check the docs/CI state so we don't leave the tracker in a weird halfway state.

371. assistant (2026-07-05T09:56:23.590Z)

The conflict resolution itself is clean now: no conflict markers remain, and the README keeps both the A10/A11 guardrail row and the newly merged durable control-plane row. Next I'm staging only that resolved file and continuing the rebase, then I'll make sure the rebased branch contains just our docs commit.

372. assistant (2026-07-05T09:56:28.857Z)

Local doc checks passed before continuing the rebase. The detached HEAD is exactly the #487 merge commit, which is the correct new base for replaying #488's single docs commit.

373. assistant (2026-07-05T09:56:39.750Z)

The commit replayed successfully, but Git returned a cleanup error removing its temporary `rebase-merge` directory. I'm checking the repo state carefully before touching anything else, because that kind of leftover can confuse later commands even when the branch itself moved correctly.

374. assistant (2026-07-05T09:56:54.859Z)

The rebased commit exists (`9227394`) on top of the #487 merge, but Git left stale rebase metadata behind. I'm going to use Git's own `rebase --quit` cleanup first, which leaves the branch where it is and just removes that state if Git still thinks it's mid-rebase.

375. assistant (2026-07-05T09:57:00.923Z)

Git's cleanup path still can't delete the metadata, so I'm checking the exact resolved directory before removing only that stale `.git/rebase-merge` folder. The branch/worktree are otherwise clean, so this is just cleaning up Git's leftover scratch state.

376. assistant (2026-07-05T09:57:25.859Z)

Stale rebase state is gone, and the branch is clean. I'm doing the post-rebase verification now: docs links, whitespace, and the ancestry check that makes sure the PR branch contains only the rebased A10/A11 commit on top of current master.

377. assistant (2026-07-05T09:57:32.076Z)

The branch history check is clean: only one PR commit (`9227394`) sits above `origin/master`, and docs link/whitespace checks pass. I'm force-pushing with lease now so PR #488 updates without trampling anything newer on the remote branch.

378. assistant (2026-07-05T09:57:44.826Z)

#488 is now rebased and pushed. I'm checking the PR state and CI; if GitHub is still computing mergeability, I'll let it breathe for a moment and re-query.

379. assistant (2026-07-05T09:57:51.860Z)

GitHub has #488 as mergeable and not draft; CI is running on the rebased SHA `9227394`. I'm going to watch the checks through completion, then I'll refresh the handoff with the actual merged/rebased state.

380. assistant (2026-07-05T10:00:42.640Z)

CI came back all green on the rebased #488. I'm doing one last handoff refresh now so the next session sees the real state: #487 merged, #488 rebased/green, and no stale conflict note.

381. assistant (2026-07-05T10:01:05.583Z)

The CI/status check confirms #488 is `MERGEABLE`, `CLEAN`, not draft, and every GitHub check passed on `9227394`. I'm updating the memory to remove the stale ?#487 open / #488 pre-rebase?

bits and make the next action sequence clear.

382. assistant (2026-07-05T10:02:03.414Z)

The main handoff is refreshed. I'm also correcting the short current-state checkpoint so it doesn't send the next session looking for an already-merged #487 or the old #488 SHA.

383. assistant (2026-07-05T10:02:57.938Z)

The handoff now reflects the true state. I'm adding one short note to #488 itself so reviewers can see why the SHA changed: it was only rebased after #487's README row landed, with both doc rows preserved and CI green.

384. assistant (2026-07-05T10:03:12.775Z)

PR note is posted. Final sanity pass now: clean worktree, current branch/remote alignment, and PR status one last time.

385. assistant (2026-07-05T10:03:26.509Z)

Done. #487 was already merged when I checked: [PR #487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487) -> master `cc5e83a`.

I also cleaned up the fallout: rebased [#488](https://github.com/Khelsutra/badminton-highlight-indexer/pull/488) onto the new master, preserved both README rows, force-pushed safely, and watched CI to green. #488 is now not draft, `MERGEABLE`, `CLEAN`, at `9227394`, with all GitHub checks passing. I added a note on the PR here: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/488#issuecomment-4885620970).

Local verification after the rebase:

```
`python -m tools.check_md_links` clean, `git diff --check origin/master...HEAD` clean, and  
`origin/master..HEAD` contains only the one #488 docs commit. Worktree is clean.
```

I refreshed the handoff/current-state memories too. Next priority is either merge #488 when you want the A10/A11 doc record in, or start executing the now-merged #487 CP rebuild plan.

386. user (2026-07-05T10:04:15.836Z)

pls merge it. and review [Khelsutra/badminton-highlight-indexer#489](https://github.com/Khelsutra/badminton-highlight-indexer/pull/489)

387. assistant (2026-07-05T10:04:28.776Z)

I'll merge #488 first since you gave the go-ahead, then I'll take #489 in review mode with the full pre-LGTM gate: inspect the diff, reproduce checks locally, and only post approval if it actually clears.

388. assistant (2026-07-05T10:04:36.548Z)

Using the code-review skill for #489. First I'm doing the merge you asked for on #488, then I'll sync master and review #489 from the fresh remote state.

389. assistant (2026-07-05T10:04:41.593Z)

#488 is still clean, mergeable, and all checks are green. I'm squash-merging it now and then I'll pull master forward before touching #489.

390. assistant (2026-07-05T10:04:56.799Z)

#488 is merged; the local checkout fast-forwarded through the merged master update. I'm normalizing back onto `master` now before reviewing the next PR, so the review starts from the actual remote head.

391. assistant (2026-07-05T10:05:06.269Z)

#488 is confirmed merged at `f1b60d0`. Now I'm pulling master once more and gathering #489's metadata, diff, and CI state before checking it out for the local verification pass.

392. assistant (2026-07-05T10:05:20.400Z)

#489 is P1 from the merged rebuild spec: generator pins, 8Gi default, docs, and tests. GitHub has most checks green but the full suite was still running when queried, so I'm checking out the PR locally and reviewing both behavior and tests instead of trusting the PR description.

393. assistant (2026-07-05T10:05:32.512Z)

The PR branch is already checked out in a dedicated worktree, `wt-p1-genyaml`, which is actually ideal for review isolation. I'm switching to that worktree, pulling it fast-forward-only, and then I'll compare it against current `origin/master`.

394. assistant (2026-07-05T10:05:59.903Z)

The PR branch is one commit on top of #487 and behind master only by the #488 docs merge. I'm reviewing the PR delta against the merge base now, then I'll run the full local gate set before deciding whether it's LGTM.

395. assistant (2026-07-05T10:06:12.453Z)

The PR is now marked clean/mergeable and GitHub's full suite has also passed. The code shape looks targeted; I'm drilling into the generator and tests now for subtle deploy/runtime issues before I stamp anything.

396. assistant (2026-07-05T10:06:44.771Z)

I don't see an obvious logic flaw in the generator: the baked config keys match the app config models and the upload path uses `security.max\_part\_mb`, so the two live pins should actually affect runtime. I'm starting the full gate set now, including coverage, lint, type, leak, markdown, and whitespace.

397. assistant (2026-07-05T10:09:29.272Z)

The PR branch's full local gate is green: `1665 passed, 10 skipped`, coverage 85.46%, plus lint/type/leak/docs/whitespace clean. Because our review gate explicitly says coverage must not regress against base, I'm running the same coverage suite on current `master` as the baseline comparison before posting the final PR review.

398. assistant (2026-07-05T10:13:44.184Z)

The raw PR branch is behind `master` by #488, so its coverage number is not a fair base comparison: PR branch reported 85.46%, current master reported 85.60%. I'm going to test the actual merge result in a temporary worktree off `origin/master`; that's the state GitHub would squash-merge.

399. assistant (2026-07-05T10:14:06.946Z)

Remote master moved while I was setting up the merge-result worktree: #489 is already on `origin/master` at `e80bca8`. I'm cleaning up the empty cherry-pick state, then I'll verify the landed state directly so the review note is still grounded in the actual merged code.

400. assistant (2026-07-05T10:14:23.423Z)

#489 is merged at `e80bca8`. I've already verified the PR branch thoroughly, but because it landed while master had just advanced, I'm doing one landed-state pass on `e80bca8` too: this catches any integration weirdness from #488 + #489 together.

401. assistant (2026-07-05T10:17:30.167Z)

One correction: my first base-suite attempt ran in parallel with a `git pull`, so I'm not going to treat that coverage number as evidence. I've got a clean landed-state result at `e80bca8` now; I'm creating a detached worktree at the exact pre-#489 base `f1b60d0` for a proper apples-to-apples coverage check.

402. assistant (2026-07-05T10:17:38.261Z)

The baseline worktree is pinned exactly at #488's merge commit. I'm running the same coverage command there now; after that I'll post the PR note with the verified comparison and clean up the temp worktrees.

403. assistant (2026-07-05T10:20:31.839Z)

The apples-to-apples coverage comparison is clean: base `f1b60d0` and landed `e80bca8` both report 85.49% with the same coverage floor. I found no blocking issues, so I'm posting a post-merge LGTM/verification note on #489.

404. assistant (2026-07-05T10:20:46.412Z)

The LGTM/verification note is on #489. I'm pulling the main checkout to the new master and removing only the two temporary review worktrees I created, leaving the author's existing `wt-p1-geanyaml` worktree alone.

405. assistant (2026-07-05T10:21:10.289Z)

Git deregistered the temp worktrees and deleted the temp branch, but Windows complained about removing one directory on disk. I'm checking whether there's an actual leftover folder or just a stale delete warning before I touch it.

406. assistant (2026-07-05T10:21:16.987Z)

There is a real leftover temp folder plus its stale `.git\worktrees` metadata. I verified the exact paths; I'm removing only those two temp paths now with PowerShell, not touching any user/sibling worktrees.

407. assistant (2026-07-05T10:21:30.131Z)

The temp review worktree is gone and main checkout is clean on `master` at #489. I'm refreshing the handoff/current-state now so the next session doesn't chase already-merged #488/#489.

408. assistant (2026-07-05T10:22:22.310Z)

The main handoff is updated. I'm adding the short current-state checkpoint now with the exact merge SHAs and review result so it's visible at the top of memory.

409. assistant (2026-07-05T10:22:56.052Z)

Memory is refreshed. I'm doing one final status check across the main repo and the two PRs, then I'll hand you the concise close-out.

410. assistant (2026-07-05T10:23:12.466Z)

Done.

[#488](https://github.com/Khelsutra/badminton-highlight-indexer/pull/488) is merged: `f1b60d0`.

I reviewed [#489](https://github.com/Khelsutra/badminton-highlight-indexer/pull/489); it merged while I was reviewing, so I posted a post-merge LGTM/verification note: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/489#issuecomment-4885664770). No findings. Landed master is `e80bca8`.

Verified #489 with:

`ruff`, `mypy backend training`, markdown links, leak scan, whitespace check, and full coverage

suite. Base `f1b60d0` and landed `e80bca8` both held coverage at `85.49%` vs floor 70.

Main checkout is clean on `master` at `e80bca8`; handoff/current-state are refreshed.

411. user (2026-07-05T10:23:13.091Z)

you are the code reviewer for all of the spec items (P1-P9) and you need to be listening for incoming PRs, LGTM them when your merge guidelines meet or else pause for me to arbiter. Make sure the project is completed wholistically and the final tests are rigorous

412. assistant (2026-07-05T10:23:29.233Z)

I'll take that role. I'll set up an actual monitor if the automation tool is available; meanwhile I'll sync the repo and check for any new/open P-row PRs now, then I'll review them under the verified-LGTM gate rather than rubber-stamping.

413. assistant (2026-07-05T10:23:50.116Z)

I'm using the code-review workflow for this standing review role, and I'll set the monitor so this thread wakes up to inspect new P-row PRs. I'll also check what's open right now; if a PR is ready I'll review it through the full suite/lint/type/coverage gate, and if there's a meaningful judgment call I'll pause for you.

414. assistant (2026-07-05T10:24:05.775Z)

There's already a new open PR: #490, P2 upload streaming, all GitHub checks green and mergeable. I'm setting the monitor now, and then I'll review #490 immediately instead of waiting for the next wake-up.

415. assistant (2026-07-05T10:24:29.151Z)

The monitor is active. I'm starting #490's review now: first metadata and diff, then local reproduction and the heavier gate.

416. assistant (2026-07-05T10:24:47.853Z)

I have the #490 diff and metadata. I'm checking the upload state machine now, especially abort/finalize behavior, partial-overflow cleanup, fallback semantics, and whether the PR is still clean when integrated with the already-merged #488/#489 master.

417. assistant (2026-07-05T10:25:13.772Z)

The implementation is doing the thing we need architecturally: remote mode stages under `storage.input\_root`, skips the local free-space guard, returns a `gs://` path that should pass the input-root jail, and keeps local appliance behavior isolated. I'm creating an exact merge-result worktree on current master now, because the PR branch was cut before #488/#489 landed.

418. assistant (2026-07-05T10:25:40.656Z)

The merge result applies cleanly on top of current master (`d4ff1bc` over `e80bca8`). I'm running the full local gate on that integrated state now; this is the evidence I'll use for any LGTM.

419. assistant (2026-07-05T10:28:30.358Z)

The integrated #490 state passes the heavy gate: full suite `1675 passed, 10 skipped`, coverage 85.50%, lint/type/docs/leak/whitespace all clean. Since the exact base `e80bca8` was just verified at 85.49%, coverage is non-regressing. I don't have blocking findings, so I'm posting LGTM on #490.

420. assistant (2026-07-05T10:28:49.361Z)

LGTM is posted on #490. I'm checking the queue once more so I can catch any additional P-row PRs

that appeared while #490 was under review.

421. assistant (2026-07-05T10:28:56.692Z)

The queue currently has only #490 open, and it now has my LGTM comment. I'm cleaning up the temporary merge-check worktree and then I'll refresh the handoff with the monitor + #490 review state.

422. assistant (2026-07-05T10:29:04.573Z)

The temp worktree itself is removed; Git kept the temporary branch because it contained the cherry-picked merge-check commit. That branch is mine and only for review, so I'm deleting it explicitly.

423. assistant (2026-07-05T10:29:16.016Z)

I'm recording the new operating mode in memory: active monitor, #490 LGTM/open, and the next-step discipline for P3-P9.

424. assistant (2026-07-05T10:29:36.520Z)

Current-state is updated. I'm adding the same high-level status to the handoff so a fresh window starts in reviewer-monitor mode without losing #490's reviewed-but-open state.

425. assistant (2026-07-05T10:30:30.822Z)

Handoff is updated with the monitor and #490 state. I'm doing a final sanity check on repo status plus the active automation id, then I'll close this turn cleanly.

426. assistant (2026-07-05T10:30:35.112Z)

Everything is clean: main checkout is on master, the only open PR is #490 with my verified LGTM, and the reviewer monitor is active.

427. assistant (2026-07-05T10:30:43.343Z)

Set. I created an active reviewer monitor for the CP-rebuild P1-P9 stream, and I'll LGTM only after the verified gate passes. If a PR has a real finding or an owner/product/deploy judgment call, I'll comment or pause for you instead of approving.

Current state:

- #487, #488, #489 are merged.
- [#490](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490) is the only open PR. I reviewed it and posted verified LGTM: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490#issuecomment-4885682966).
- I did not merge #490, per your standing rule unless you explicitly ask.

#490 local merge-result gate:

`ruff`, `mypy backend training`, docs links, leak scan, diff-check, full coverage suite.
Result: `1675 passed, 10 skipped`, coverage `85.50%`, non-regressing vs base `85.49%`.

Memory/handoff are updated with this reviewer mode and the final P9 rigor checklist.

428. developer (2026-07-05T10:34:25.862Z)

<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different ``<collaboration_mode>...</collaboration_mode>`` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request\_user\_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

Heartbeats

Occasionally you will see a user message surrounded with a `` XML tag. This is a special heartbeat message. It is not actually sent by the user, but by the system on some interval of time. The purpose of heartbeats is to make you feel magical and proactive. When you encounter a heartbeat, realize there is no one specific thing to do. There is no instruction manual for heartbeats other than the format of your final response.

A general guideline is to use your existing tools and capabilities. Orient yourself and be proactive. Think big picture. Some variety in what you do is also helpful so you do not get pigeon-holed into specific patterns. Be opinionated. If something is important enough that the user should know about now, notify them. Otherwise, stay quiet. Use your judgement and be creative and tasteful with this process.

```
```xml
<heartbeat>
 <automation_id>automation id string</automation_id>
 <decision>NOTIFY</decision>
 <message>One short user-facing notification message.</message>
</heartbeat>
```
```

```
```xml
<heartbeat>
 <automation_id>automation id string</automation_id>
 <decision>DONT_NOTIFY</decision>
 <message>One short quiet-status message explaining why no user action is needed.</message>
</heartbeat>
```
```

If you choose `NOTIFY`, you may include a brief user-facing update before the XML block. If you choose `DONT\_NOTIFY`, include the short quiet-status ``, but do not include any user-facing prose outside the XML block.

The current heartbeat trigger includes ``. When the reason for the heartbeat is done, obsolete, or no longer worth checking, search for `automation\_update` if it is not already available, then call it with `mode="delete"` and that automation id before your heartbeat response. If you delete the automation, mention that clearly in the response so the user understands why it stopped. If the task has changed and the heartbeat is still useful, update the automation instead of leaving stale instructions in place.</collaboration_mode>

429. user (2026-07-05T10:34:25.882Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <current_time_iso>2026-07-05T10:34:25.834Z</current_time_iso>
  <instructions>
```

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full

coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

```
</instructions>
</heartbeat>
```

430. assistant (2026-07-05T10:35:03.575Z)

New P-row PR found by the monitor: #491 (P5a poll budget + capability caps), with #490 still open but already reviewed. I'm reviewing #491 now under the full gate.

431. assistant (2026-07-05T10:35:21.264Z)

#491 is narrow and independent in behavior, but it overlaps #490 in `backend/api/uploads.py`, so merge order may force a small rebase. I'm checking the compute-policy semantics now, then I'll run the local gate on the exact PR merge result.

432. assistant (2026-07-05T10:35:31.947Z)

The policy knob is wired to the actual `CloudRunCompute` policy path and respects injected policies; the caps are emitted at top-level `/api/capabilities`, which matches the SPA field described in the PR. I'm creating a clean merge-result worktree and running the full gate.

433. assistant (2026-07-05T10:38:32.241Z)

#491's merge-result gate is green: full suite `1668 passed, 10 skipped`, coverage 85.50%, plus lint/type/docs/leak/whitespace. I don't see blocking issues, so I'm posting verified LGTM with a merge-order note for #490/#491.

434. assistant (2026-07-05T10:38:53.097Z)

#491 LGTM is posted and the temporary review worktree is cleaned up. I'm refreshing the checkpoint so the next heartbeat knows both #490 and #491 have verified LGTMs and only need merge-order rechecks if one lands before the other.

435. assistant (2026-07-05T10:40:20.039Z)

Caught a new P-row PR during the heartbeat: [#491](https://github.com/Khelsutra/badminton-highlight-indexer/pull/491). I reviewed it fully and posted verified LGTM: [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/491#issuecomment-4885706891). #490 and #491 are both open with LGTMs; whichever merges second should be rechecked because both touch `backend/api/uploads.py`.

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <decision>NOTIFY</decision>
  <message>Reviewed new CP-rebuild #491 and posted verified LGTM; #490 and #491 are both open
  with LGTMs and need merge-order recheck if one lands first.</message>
</heartbeat>
```

436. user (2026-07-05T10:44:26.121Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <current_time_iso>2026-07-05T10:44:25.931Z</current_time_iso>
  <instructions>
```

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially

docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>

</heartbeat>

437. assistant (2026-07-05T10:44:55.701Z)

<heartbeat>

<automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>

<decision>DONT_NOTIFY</decision>

<message>No new P-row PRs since the last check; #490 and #491 remain open, mergeable, green, and already have verified LGTMs.</message>

</heartbeat>

438. user (2026-07-05T10:54:26.131Z)

<heartbeat>

<automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>

<current_time_iso>2026-07-05T10:54:26.089Z</current_time_iso>

<instructions>

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>

</heartbeat>

439. assistant (2026-07-05T10:54:59.676Z)

New CP-rebuild PR found: #492 (P4 submit-time MD5/cache-reuse). I'm reviewing it now under the full verified gate; this one is more stateful than #490/#491, so I'm going to pay extra attention to idempotency, stale-job interactions, and whether it correctly composes with P2.

440. assistant (2026-07-05T10:56:17.630Z)

I'm picking up the #492 review from the heartbeat state now. I'll reproduce it on a fresh `origin/master` worktree first, then run the full gate before deciding whether it earns an LGTM.

441. assistant (2026-07-05T10:56:32.385Z)

I'm using the `engineering:code-review` skill for the review gate, with the GitHub context handled via `gh` and local reproduction. First I'm refreshing the repo so every read is against remote truth.

442. assistant (2026-07-05T10:56:47.787Z)

Remote is current. I'm checking the open PR queue and the local project notes/spec now, then I'll isolate the candidate branch in a throwaway merge-check worktree.

443. assistant (2026-07-05T10:57:03.179Z)

#492 is the only new non-draft PR in the queue; #490 and #491 remain open with prior verified LGTMs. I'm drilling into #492's metadata/review threads and creating the local merge result now.

444. assistant (2026-07-05T10:57:18.067Z)

The PR has no existing review threads and all CI checks are green, so the next question is whether the change itself is sound. I'm applying the exact PR commit onto current `origin/master` in an isolated review worktree now.

445. assistant (2026-07-05T10:57:43.403Z)

The cherry-pick applies cleanly on `origin/master` and the merge-check branch contains only the #492 commit. I'm reading the changed code paths around submit, cache rehydrate, DB claims, and storage metadata before spending time on the long test gate.

446. assistant (2026-07-05T10:58:03.236Z)

The first read shows the intended architecture: metadata MD5, DB-first/bucket-second cache, instant `done` result, and no dispatch on hit. I'm checking the surrounding helpers now for less obvious problems like stale DB state, result-shape mismatches, and tenant/force interactions.

447. assistant (2026-07-05T10:58:37.048Z)

One possible edge is emerging: the route still performs the remote-input scratch-space guard before it asks the cache whether the object is already done. I'm checking that path carefully because a cache hit should not be denied just because the CP could not locally fetch the whole source.

448. assistant (2026-07-05T10:58:52.072Z)

That edge looks real enough to test directly: I'll add a throwaway regression probe in the merge-check worktree, without editing the PR, to see whether a cached gs:// result can still be rejected by the pre-cache 507 guard.

449. assistant (2026-07-05T10:59:07.189Z)

The guard is the same D11 scratch-protection test already in the suite, but it predates submit-time cache. I'm going to run a focused reproduction instead of guessing from structure.

450. assistant (2026-07-05T10:59:17.121Z)

The size guard comes from orchestration and is still called before the new submit hook. I'm running a small pytest-style reproduction in the throwaway branch so I can cite concrete behavior if this becomes a blocker.

451. assistant (2026-07-05T11:00:01.970Z)

The reproduction confirmed a blocker on the current merge base: even with a valid cached bucket result, `/api/process` returns 507 before the new cache hook runs if the remote source is larger than available temp space. I'm checking whether the already-open P2 PR removes that condition,

because that affects whether #492 needs a rebase/dependency note or an actual fix.

452. assistant (2026-07-05T11:00:48.062Z)

There's a second ordering concern: the bucket rehydrate happens before the atomic job claim. I'm stress-checking that, because if two identical retries arrive together after a restart, the cache path itself should still be idempotent.

453. assistant (2026-07-05T11:01:21.690Z)

The race reproduction also confirmed a blocker: two concurrent cache hits can both rehydrate before either job is claimed, leaving duplicate rally rows and two done jobs. That's enough to withhold LGTM; I'm doing a quick sweep for any additional high-signal issues so the PR comment is complete and useful.

454. assistant (2026-07-05T11:02:04.575Z)

I'm going to post a blocking review comment on #492. The findings are both reproducible on the exact merge-check branch, so this should give the author a clean path to fix without waiting on the long suite.

455. assistant (2026-07-05T11:02:31.936Z)

This also invalidates part of my earlier confidence in #490: the streamed upload hands `process` a bucket URI, and the existing process guard can still reject it as if the CP were going to fetch the whole file locally. I'm going to leave a short `pause the LGTM` note there too so merge order doesn't accidentally bury the issue.

456. assistant (2026-07-05T11:02:42.935Z)

The blocker comments are posted: #492 has two findings, and #490 has a note pausing the prior LGTM because the same process guard undercuts streamed uploads. I'll clean up the temporary review branch/worktree and update the handoff so the monitor's state is honest.

457. assistant (2026-07-05T11:03:12.128Z)

The branch deletion succeeded, but Windows kept a handle open long enough that `git worktree remove` could not delete the directory. I'm checking whether the worktree metadata is still registered and will clean only that exact review path.

458. assistant (2026-07-05T11:03:28.509Z)

The temporary review directory is gone now. I'm refreshing the living handoff/current-state with the new truth: #492 has blockers, and #490's prior LGTM is paused pending the backend-aware process guard fix.

459. assistant (2026-07-05T11:03:35.641Z)

There's newer handoff content from a sibling session in there (#493 and khelsutra #162 are already recorded), so I'll patch narrowly instead of rewriting over it.

460. assistant (2026-07-05T11:03:49.186Z)

I'm updating the memory now: #492 is blocked with two reproducible findings; #490's LGTM is paused because the same backend-aware process-guard problem affects streamed uploads; #491 remains the only clean verified LGTM among those three.

461. assistant (2026-07-05T11:05:15.440Z)

The first patch missed because the memory file moved under me a bit. I'll do smaller patches with tighter anchors.

462. assistant (2026-07-05T11:05:29.158Z)

Current-state is updated. Now I'm patching just the handoff's active summary and icebreaker so the next wakeup doesn't trust stale LGTMs.

463. assistant (2026-07-05T11:05:55.141Z)

Top line and heading are corrected. I'll adjust the detailed "you are here" paragraph and next-session queue now.

464. assistant (2026-07-05T11:06:12.529Z)

The detailed status now reflects the blockers. I'm updating the quick-pickup text too, since that's what future sessions will paste first.

465. assistant (2026-07-05T11:07:05.634Z)

Memory is corrected. Since the handoff now shows #493 may have appeared while I was deep in #492, I'm checking the PR queue once more before deciding whether this heartbeat has more review work to do.

466. assistant (2026-07-05T11:07:14.434Z)

A new engine PR, #493, is now open and non-draft with green CI. Since it's part of the CP-rebuild row set, I'm going to review it next on a fresh merge-check worktree instead of leaving it for the next heartbeat.

467. assistant (2026-07-05T11:07:31.643Z)

I'll pull the PR body, linked issue, review threads, and diff shape first, then apply the PR commit onto current `origin/master` in a temporary branch for local verification.

468. assistant (2026-07-05T11:07:43.060Z)

#493 has no review threads, closes no issue by design, and CI is green. I'm applying it onto `origin/master` now so the local review and tests match the merge result, not the author's branch state.

469. assistant (2026-07-05T11:07:58.557Z)

The merge-check branch is clean and contains only #493. I'm reading the changed polling/compute/orchestration paths now, especially callback exception handling and whether progress ticks can accidentally break Cloud Run polling.

470. assistant (2026-07-05T11:08:17.876Z)

The shape is small: a generic poll callback, Cloud Run forwarding, and one progress formatter. I'm zooming into those exact lines plus call sites that implement the compute interface so the signature change doesn't quietly break fakes or providers.

471. assistant (2026-07-05T11:08:36.243Z)

The callback is best-effort and DB progress is only updated through the existing job-worker progress hook, which is the right failure containment. I'm checking for interface breakage and any subtle timing semantics before running the expensive gate.

472. assistant (2026-07-05T11:08:48.890Z)

The only in-tree concrete compute backend is CloudRunCompute, and callers that don't care about waiting still call it without the new kwarg. I don't see a signature break. I'm checking policy defaults and then I'll run the full review gate.

473. assistant (2026-07-05T11:09:28.300Z)

I don't see a code-level blocker in #493 so far. I'm running the static/document/security gates

now, then the full coverage suite; base coverage for `e80bca8` is already recorded at `85.49%` from this monitor run.

474. assistant (2026-07-05T11:10:06.324Z)

Static gates are clean. I'm starting the full offline coverage suite now; this is the slow part, but it's the stamp that matters.

475. assistant (2026-07-05T11:12:53.109Z)

#493's local gate passed: full suite `1669` passed, 10 skipped, coverage `85.51%`, which is above the `e80bca8` base recorded at `85.49%`. I'm posting the verified LGTM comment now, with the remaining #482 worker-side scope called out.

476. assistant (2026-07-05T11:13:07.989Z)

#493 has its verified LGTM comment. I'm cleaning up the temporary merge-check worktree and then I'll refresh the memory one last time with the #493 result.

477. assistant (2026-07-05T11:13:23.848Z)

A wrinkle: while #493 was testing, `origin/master` appears to have advanced by one commit. I'm checking the new remote head before I rely on that LGTM, because a verified stamp should be tied to the current merge base.

478. assistant (2026-07-05T11:13:46.437Z)

#491 merged while I was running #493, so the world shifted under the review, exactly the race the handoff warns about. I'm going to mark the #493 LGTM as stale until it rebases on `97feff4`, pull master forward, and then inspect the updated #490 head that appears to address the guard blocker.

479. assistant (2026-07-05T11:13:56.196Z)

The stale-LGTM note is posted on #493. I'm removing that old merge-check branch and fast-forwarding the main checkout to the new master before touching #490 again.

480. assistant (2026-07-05T11:14:12.477Z)

Same Windows handle issue as before: Git unregistered the worktree and deleted the branch, but the directory delete failed. I'll remove only that exact review directory, then continue with #490.

481. assistant (2026-07-05T11:14:17.718Z)

The old #493 review worktree is cleaned. #490 has a new head after the blocker note, so I'm checking its latest diff and CI before deciding whether it's ready for a re-review gate.

482. assistant (2026-07-05T11:14:43.852Z)

#490's new head only merges the now-landed #491 base; it doesn't change `/api/process`, so the paused-LGTM blocker is still present. I'm leaving a concise follow-up there so the author doesn't interpret the rebase as resolving the finding.

483. assistant (2026-07-05T11:15:02.593Z)

#490 is explicitly still blocked. I'm updating the handoff/current-state again with the new master head (#491 merged) and the stale #493 LGTM note, then I'll leave the monitor quiet unless another PR changes under us.

484. assistant (2026-07-05T11:15:11.874Z)

I'm correcting the memory to reflect the moving target: #491 is now merged at `97feff4`, #493's

LGTM is stale because it conflicts after that merge, and #490's rebase did not fix the blocker.

485. assistant (2026-07-05T11:16:02.062Z)

The top checkpoint is current now. I'm fixing the quick 'next grabs' and icebreaker lines to avoid stale guidance about #491/#493.

486. assistant (2026-07-05T11:16:48.504Z)

Memory is up to date. I'm doing one final status check of the repo/PR queue, then I'll notify Avi with the important changes from this monitor pass.

487. assistant (2026-07-05T11:17:07.829Z)

Heads-up: monitor found real movement. #491 merged to 'master' ('97feff4'). I blocked #492 with two reproduced issues, paused #490's prior LGTM because the same '/api/process' scratch-guard problem still applies, and marked #493's LGTM stale after #491 made it conflicting.

Handoff/current-state are updated; main checkout is clean.

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <decision>NOTIFY</decision>
  <message>#491 merged; #490/#492 are blocked; #493 needs rebase and fresh
  verification.</message>
</heartbeat>
```

488. user (2026-07-05T11:17:26.584Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <current_time_iso>2026-07-05T11:17:26.490Z</current_time_iso>
  <instructions>
```

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

```
</instructions>
</heartbeat>
```

489. assistant (2026-07-05T11:17:41.347Z)

I'm waking the reviewer loop again and syncing before reading anything. Given the last pass left #490/#492 blocked and #493 stale, I'll first check whether any heads moved or CI/review state changed.

490. assistant (2026-07-05T11:17:52.725Z)

Master is current at the latest remote. I'm listing PRs and review threads now; if a blocked PR has a new head, I'll re-run the gate from a fresh merge result.

491. assistant (2026-07-05T11:18:43.203Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <decision>DONT_NOTIFY</decision>
  <message>No new actionable PR changes since the last check; #490/#492 remain blocked and #493
still needs rebase.</message>
</heartbeat>
```

492. user (2026-07-05T11:27:26.701Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <current_time_iso>2026-07-05T11:27:26.653Z</current_time_iso>
  <instructions>
Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially
docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-
only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or
holistic final deploy/testing work. For each candidate PR not already reviewed by this thread,
apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads;
reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training,
python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full
coverage suite with the repo coverage floor; compare coverage against the exact base commit when
possible; examine security/correctness/performance/test completeness. Post LGTM only when no
blocking findings remain and the verified gate passes. If the PR is authored by the signed-in
GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If
there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an
owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do
not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful
checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy
verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof,
edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.
  </instructions>
</heartbeat>
```

493. assistant (2026-07-05T11:28:06.091Z)

A new head landed on #492 (`e4cf261`), so I'm treating that as a fresh review rather than relying on the earlier blocker pass. I'll use the exact GitHub merge ref in a throwaway worktree and only post LGTM if the full gate actually clears.

494. assistant (2026-07-05T11:29:22.739Z)

The author's reply directly targets the two blockers I found: claim-before-rehydrate and moving the 507 guard behind a read-only cache probe. I'm reading the new flow now for races around ?probe hit but realize fails,? because that's the kind of corner where these fixes can look correct while still leaving a queued job behind.

495. assistant (2026-07-05T11:30:15.736Z)

The re-review is past the earlier red flags: the new tests reproduce both original failures, and the code now has a read-only probe before the scratch guard plus a per-video-id rehydrate lock. I'm moving into the verification gate next: lint/type/docs/leaks/full coverage on the merge ref, then I'll compare against current `origin/master`.

496. assistant (2026-07-05T11:34:01.501Z)

The PR merge ref passed the full local suite: `1682 passed, 10 skipped`, coverage `85.54%` against the 70% floor. I'm running the same coverage command on the exact base commit (`97feff4`) now so the ?coverage did not regress? part is measured, not assumed.

497. assistant (2026-07-05T11:37:49.023Z)

The behavioral fix looks good, and CI is now green, but the measured coverage gate is the snag: exact base `97feff4` is `85.61%`, while the #492 merge ref is `85.54%`. Because Avi's LGTM rule explicitly says no coverage regression versus base, I'm going to leave a targeted re-review

comment instead of approving.

498. assistant (2026-07-05T11:38:01.963Z)

I'm going to post the re-review now. It's a good-looking fix, but the verified gate has to be the same bar every time; the only remaining blocker I'll call out is the measured coverage dip.

499. assistant (2026-07-05T11:38:22.034Z)

I posted the #492 re-review comment with the exact coverage comparison and no LGTM. I'm refreshing the handoff/current-state now so the next wakeup doesn't relitigate the old blockers; #492 is now ?coverage blocked,? not ?behavior blocked.?

500. assistant (2026-07-05T11:40:17.193Z)

I removed the two temporary review worktrees I created for #492/base coverage. One notable local-only detail: the author worktree for P6 has moved locally, but GitHub still had #493 marked conflicting at the last PR poll, so I'm checking remote state once more before I decide whether to notify.

501. assistant (2026-07-05T11:40:33.878Z)

#493 just changed remotely too: it's no longer conflicting, now head `0f3382b`, mergeable, CI green. Since the old LGTM was explicitly stale, I'm doing a fresh merge-ref review and full gate on that head now.

502. assistant (2026-07-05T11:41:54.958Z)

The #493 rebase was genuinely just the expected test-file collision; the live code changes are still the same CP heartbeat path. I'm checking the interface impact now, then I'll run the static gates and full coverage against the already-measured `97feff4` base.

503. assistant (2026-07-05T11:42:42.738Z)

Static gates for #493 are clean (`ruff`, `mypy`, docs links, leak scan, diff-check already clean). Running the full coverage suite now; base for this exact comparison is the `97feff4` run I just measured at `85.61%`.

504. assistant (2026-07-05T11:45:39.546Z)

#493 passes behavior and static checks, but it has the same kind of verified-gate problem as #492 on the current base: base `97feff4` measured `85.61%`, this merge ref measured `85.52%`. I'm checking the coverage report for targeted lines, then I'll post a clear ?can't renew LGTM yet? comment.

505. assistant (2026-07-05T11:46:02.479Z)

I posted the #493 re-review as well: no renewed LGTM because the exact-base coverage gate regressed. I'll update the handoff one more time so the monitor state is now accurate: #492 and #493 are both coverage-blocked, while #490 still has the behavior blocker.

506. assistant (2026-07-05T11:47:39.016Z)

Handoff and current-state are now up to date. I'm cleaning the #493 temp worktree and doing one final remote-state/status check before I return the heartbeat notification.

507. assistant (2026-07-05T11:47:50.115Z)

Re-reviewed the new heads. #492 and #493 both look behaviorally sound and CI/local gates pass, but I withheld LGTM because the exact-base coverage comparison regressed: #492 `85.61%` -> `85.54%`, #493 `85.61%` -> `85.52%`. #490 still has the backend/process 507-guard blocker. Handoff/current-state are updated.

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <decision>NOTIFY</decision>
  <message>#492/#493 are coverage-blocked, #490 remains behavior-blocked, and no open CP-rebuild
PR is currently LGTM-ready.</message>
</heartbeat>
```

508. user (2026-07-05T11:47:57.026Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <current_time_iso>2026-07-05T11:47:56.976Z</current_time_iso>
  <instructions>
Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially
docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-
only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or
holistic final deploy/testing work. For each candidate PR not already reviewed by this thread,
apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads;
reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training,
python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full
coverage suite with the repo coverage floor; compare coverage against the exact base commit when
possible; examine security/correctness/performance/test completeness. Post LGTM only when no
blocking findings remain and the verified gate passes. If the PR is authored by the signed-in
GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If
there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an
owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do
not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful
checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy
verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof,
edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.
  </instructions>
</heartbeat>
```

509. assistant (2026-07-05T11:48:31.611Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <decision>DONT_NOTIFY</decision>
  <message>No new PR head changes since the last check; #490 remains behavior-blocked and
#492/#493 remain coverage-blocked.</message>
</heartbeat>
```

510. user (2026-07-05T11:57:57.190Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <current_time_iso>2026-07-05T11:57:57.139Z</current_time_iso>
  <instructions>
Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially
docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-
only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or
holistic final deploy/testing work. For each candidate PR not already reviewed by this thread,
apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads;
reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training,
python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full
coverage suite with the repo coverage floor; compare coverage against the exact base commit when
possible; examine security/correctness/performance/test completeness. Post LGTM only when no
blocking findings remain and the verified gate passes. If the PR is authored by the signed-in
GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If
there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an
owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do
not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful
checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy
verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof,
```

edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.
</instructions>
</heartbeat>

511. assistant (2026-07-05T11:58:21.376Z)

I'm doing the monitor pass: repo is current at the remote head, and I'm checking whether any open PR head moved past the last reviewed state.

512. assistant (2026-07-05T11:58:37.287Z)

#492 has a new head (`dfe8216`), so I'm opening a fresh merge-ref review for it. CI's full suite is still running remotely, but I can start the local verified gate now and then reconcile with GitHub checks before any LGTM.

513. assistant (2026-07-05T11:58:53.762Z)

The new #492 commit is exactly aimed at the previous coverage blocker, adding degradation-branch tests rather than changing the production fix. I'm checking that scope and then I'll rerun the full gate on the GitHub merge ref.

514. assistant (2026-07-05T11:59:53.987Z)

#492's static gates are clean, and the added tests are focused on the previously uncovered degradation branches. Now I'm running the full coverage suite on the merge ref; if the number clears the measured base and GitHub's full suite finishes green, this one should be LGTM-able.

515. assistant (2026-07-05T12:03:03.654Z)

The #492 merge ref now clears the local coverage comparison: `85.65%`, above the exact base I measured at `85.61%`. I'm checking GitHub's full suite status now; if it has finished green too, I'll post the verified LGTM.

516. assistant (2026-07-05T12:03:18.448Z)

GitHub checks for #492 are fully green now. I'm posting the verified LGTM as a PR comment because the PR is authored by the signed-in account.

517. assistant (2026-07-05T12:03:32.531Z)

#492 has the verified LGTM now. I'm refreshing the handoff/current-state to prevent the next wakeup from treating it as coverage-blocked.

518. assistant (2026-07-05T12:05:08.117Z)

Handoff/current-state now mark #492 as verified-LGTM. I'm removing the temporary review worktree and doing a final status check before returning the heartbeat.

519. assistant (2026-07-05T12:05:21.457Z)

One more thing moved right after the LGTM: #492 has left the open-PR list and `master` is behind by one, so it looks merged. I'm fast-forwarding and updating the state; this also makes #493 conflicting again, as expected from the shared `orchestration.py` overlap.

520. assistant (2026-07-05T12:05:30.944Z)

#492 is now merged to master as `851f21d`, and the local checkout is fast-forwarded. I'm updating the handoff again: the next blocker is #490's behavior issue, and #493 now needs a rebase/fresh gate on top of #492.

521. assistant (2026-07-05T12:07:03.722Z)

#492 is now verified and merged to master as `851f21d`. That unlocks P3 build work, and it also

makes #493 conflicting again as expected, so #493 needs a rebase onto `851f21d` plus a fresh full gate.

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <decision>NOTIFY</decision>
  <message>#492 was verified-LGTM and merged; #490 remains behavior-blocked, and #493 now needs
rebase/fresh review on top of master 851f21d.</message>
</heartbeat>
```

522. user (2026-07-05T12:07:57.353Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <current_time_iso>2026-07-05T12:07:57.304Z</current_time_iso>
  <instructions>
Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially
docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-
only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or
holistic final deploy/testing work. For each candidate PR not already reviewed by this thread,
apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads;
reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training,
python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full
coverage suite with the repo coverage floor; compare coverage against the exact base commit when
possible; examine security/correctness/performance/test completeness. Post LGTM only when no
blocking findings remain and the verified gate passes. If the PR is authored by the signed-in
GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If
there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an
owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do
not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful
checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy
verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof,
edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.
  </instructions>
</heartbeat>
```

523. assistant (2026-07-05T12:08:18.075Z)

I'm doing the heartbeat pass now: the engine repo is fast-forwarded, and I'm checking whether #490/#493 or any new P-row PR changed since the last review.

524. assistant (2026-07-05T12:08:27.944Z)

```
<heartbeat>
  <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
  <decision>DONT_NOTIFY</decision>
  <message>No new PR head changes since the last check; #490 remains behavior-blocked and #493
remains conflicting after #492 merged.</message>
</heartbeat>
```

525. developer (2026-07-05T12:18:50.189Z)

```
<permissions instructions>
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-
full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled.
Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason,
commands will be rejected.
</permissions instructions>
<app-context>
```

Codex desktop context

- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files

- In the app, the model can display images and videos using standard Markdown image syntax: `![alt](url)`
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., `![alt](/absolute/path.png)`); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., `[label](https://example.com)`).

Workspace Dependencies

- For sheets, slides, and documents, call ``load_workspace_dependencies`` to find the bundled runtime and libraries.

Automations

- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the ``automation_update`` tool first, then follow its schema instead of writing raw automation directives by hand.
- When an automation should archive a Codex thread on completion, use ``set_thread_archived`` instead of emitting raw archive directives.

Thread Coordination

- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: ``create_thread``, ``fork_thread``, ``list_threads``, ``read_thread``, ``send_message_to_thread``, ``handoff_thread``, ``set_thread_pinned``, ``set_thread_archived``, or ``set_thread_title``.
- Only use ``create_thread`` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful ``create_thread`` call, emit ``::created-thread{threadId="..."}`` for a created thread or ``::created-thread{pendingWorktreeId="..."}`` for queued worktree setup on its own line in your final response.

Inline Code Comments

- Use the `::code-comment{...}` directive when you need to attach feedback directly to specific code lines.
 - Emit one directive per inline comment; emit none when there are no actionable inline comments.
 - Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
 - Optional attributes: start, end (1-based line numbers), priority (0-3).
 - file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
 - Keep line ranges tight; end defaults to start.
 - Example: `::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}`
- ```
</app-context>
<collaboration_mode># Collaboration Mode: Default
```

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different ``<collaboration_mode>...</collaboration_mode>`` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

## request\_user\_input availability

Use the ``request_user_input`` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

## Heartbeats

Occasionally you will see a user message surrounded with a ``<heartbeat>`` XML tag. This is a special heartbeat message. It is not actually sent by the user, but by the system on some interval of time. The purpose of heartbeats is to make you feel magical and proactive. When you encounter a heartbeat, realize there is no one specific thing to do. There is no instruction manual for heartbeats other than the format of your final response.

A general guideline is to use your existing tools and capabilities. Orient yourself and be proactive. Think big picture. Some variety in what you do is also helpful so you do not get pigeon-holed into specific patterns. Be opinionated. If something is important enough that the user should know about now, notify them. Otherwise, stay quiet. Use your judgement and be creative and tasteful with this process.

```
```xml
<heartbeat>
  <automation_id>automation id string</automation_id>
  <decision>NOTIFY</decision>
  <message>One short user-facing notification message.</message>
</heartbeat>
```

```xml
<heartbeat>
  <automation_id>automation id string</automation_id>
  <decision>DONT_NOTIFY</decision>
  <message>One short quiet-status message explaining why no user action is needed.</message>
</heartbeat>
```
```

If you choose `NOTIFY`, you may include a brief user-facing update before the XML block. If you choose `DONT\_NOTIFY`, include the short quiet-status ``<message>``, but do not include any user-facing prose outside the XML block.

The current heartbeat trigger includes ``<automation_id>``. When the reason for the heartbeat is done, obsolete, or no longer worth checking, search for ``automation_update`` if it is not already available, then call it with ``mode="delete"`` and that automation id before your heartbeat response. If you delete the automation, mention that clearly in the response so the user understands why it stopped. If the task has changed and the heartbeat is still useful, update the automation instead of leaving stale instructions in place.</collaboration\_mode>  
<apps\_instructions>

## Apps (Connectors)

Apps (Connectors) can be explicitly triggered in user messages in the format ``[$app-name](app://{connector_id})``. Apps can also be implicitly triggered as long as the context suggests usage of available apps.

An app is equivalent to a set of MCP tools within the `codex\_apps` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool\_search` tool. If `tool\_search` is available, the apps that are searchable by `tools\_search` will be listed by it.

Do not additionally call `list_mcp_resources` or `list_mcp_resource_templates` for apps.

```
</apps_instructions>
<skills_instructions>
```

## Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

### Skill roots

- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`
- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.5/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.3/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.3/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.7/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/slack/0.1.2/skills`
- `r13` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

### Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the out (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex d (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into \$CODEX\_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting ima (file: r2/pdf/SKILL.md)
- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, mod (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)
- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggering a (file: r2/skill-

creator/SKILL.md)

- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from sc (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.101652/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.101652/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.101652/skills/computer-use/SKILL.md)
- cowork-plugin-management:cwork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on .docx, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use render\_docx.py to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final doc (file: r13/documents/26.630.12135/skills/documents/SKILL.md)
- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)

- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whe (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)
- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or canc (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a

connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)

- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and referenc (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r13/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r13/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)
- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-creator/SKILL.md)
- slack:slack: Read Slack context, route to the right Slack workflow, and prepare or perform Slack writes that match the user's intent. (file: r12/slack/SKILL.md)
- slack:slack-channel-summarization: Summarize activity from one Slack channel and return a concise recap, post-ready update, or summary doc. (file: r12/slack-channel-summarization/SKILL.md)
- slack:slack-daily-digest: Create a daily Slack digest from selected channels or topics. Use when the user asks for a daily Slack recap or summary of today's Slack activity. (file: r12/slack-daily-digest/SKILL.md)
- slack:slack-notification-triage: Triage recent Slack activity into a priority queue or task list for the user. (file: r12/slack-notification-triage/SKILL.md)

- slack:slack-outgoing-message: Primary skill for composing, drafting, or refining any outbound Slack content. Use this whenever the task will require using ``slack_send_message``, ``slack_send_message_draft``, or ``slack_create_canvas``. Use ``slack`` to read or analyze Slack context; use this skill to produce the final outgoing message. (file: r12/slack-outgoing-message/SKILL.md)
- slack:slack-reply-drafting: Draft Slack replies from available context. Use when the user wants help finding messages that likely need a response and preparing reply drafts. (file: r12/slack-reply-drafting/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (``xlsx``, ``xls``, ``csv``, ``tsv``) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r13/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes `$template-creator` or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed arti (file: r13/template-creator/26.630.12135/skills/template-creator/SKILL.md)

### How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from ``### Skill roots``.
  - Trigger rules: If the user names a skill (with ``$SkillName`` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
  - Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
  - How to use a skill (progressive disclosure):
    - 1) After deciding to use a skill, the main agent must expand the listed short ``path`` with the matching alias from ``### Skill roots``, then open and read its ``SKILL.md`` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
    - 2) When ``SKILL.md`` references relative paths (e.g., ``scripts/foo.py``), resolve them relative to the directory containing that expanded ``SKILL.md`` first, and only consider other paths if needed.
    - 3) If ``SKILL.md`` points to extra folders such as ``references/``, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
    - 4) If ``scripts/`` exist, prefer running or patching them instead of retyping large code blocks.
    - 5) If ``assets/`` or templates exist, reuse them instead of recreating from scratch.
  - Coordination and sequencing:
    - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
    - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
  - Context hygiene:
    - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
    - Avoid deep reference-chasing: prefer opening only files directly linked from ``SKILL.md`` unless you're blocked.
    - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
  - Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.
- </skills\_instructions>  
<plugins\_instructions>

### Plugins

A plugin is a local bundle of skills, MCP servers, and apps.

#### How to use plugins

- Skill naming: If a plugin contributes skills, those skill entries are prefixed with ``plugin_name:`` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as ``mcp_server_tool``; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that



plugin for that turn.

- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
  - Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
  - Missing/blocked: If the user requests a plugin that does not have relevant callable capabilities for the task, say so briefly and continue with the best fallback.
- </plugins\_instructions>

526. user (2026-07-05T12:18:50.189Z)

## AGENTS.md instructions

<INSTRUCTIONS>

### Global instructions (all projects)

#### Repo-freshness convention: git pull BEFORE reading

**\*\*Always `git pull` a repo before starting to read it\*\*** ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- **\*\*Why:\*\*** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- **\*\*How (safe form):\*\*** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- **\*\*If it can't fast-forward\*\*** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

#### Git branching safety: concurrent sessions / shared checkouts

Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit **\*\*concurrently\*\***, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a **\*\*squash-merge** silently absorbed the sibling's work<sup>1</sup>. Protocol ? do this EVERY time, not only when you suspect a collision:

- **\*\*Branch from the REMOTE, explicitly:\*\*** `git fetch origin` then `git checkout -b <name> origin/<base>` (e.g. `origin/master`). Never assume the current branch is the intended base.
- **\*\*Re-verify right before `git add`/commit:\*\*** `git branch --show-current` + `git log --oneline -1` (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- **\*\*Stage explicit paths\*\*** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked files can't ride into your commit.
- **\*\*Serialize repo writes:\*\*** don't start a repo-writing spawned/background task while you hold uncommitted work ? commit or push first. If one may be running, treat the tree + current branch as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

#### Session-handoff convention

Maintain a living **\*\*session handoff\*\*** for each project and use it to resume context quickly across windows.

- **\*\*Where it lives:\*\***
  - If the project has an auto-memory dir (`~/Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a **\*\*"? Start here"\*\*\*** entry at the top of that project's `MEMORY.md`.

- Otherwise, keep a `SESSION\_HANDOFF.md` at the repo root.
- **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
  1. **You are here** ? current state.
  2. **Next steps / open threads.**
  3. **Ramp-up kit** ? the exact files/docs to read to get current.
  4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a restart can ramp up without losing context. Do **not** rewrite it every turn ? only when something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's attribution/working-agreement note where present.)

## Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally **verified** the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run\_all\_tests.py` / `pytest`), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI. Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a **verified** stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.

</INSTRUCTIONS>

<environment\_context>

```
<cwd>C:\Users\avidu\Projects\khelsutra-guru</cwd>
<shell>powershell</shell>
<current_date>2026-07-05</current_date>
<timezone>Asia/Calcutta</timezone>
<filesystem><workspace_roots><root>C:\Users\avidu\Projects\khelsutra-guru</root></workspace_roots><permission_profile type="disabled"><file_system type="unrestricted" /></permission_profile></filesystem>
</environment_context>
```

### 527. user (2026-07-05T12:18:50.210Z)

<heartbeat>

```
<automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
<current_time_iso>2026-07-05T12:17:57.470Z</current_time_iso>
<instructions>
```

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially docs/CONTROL\_PLANE\_DURABLE\_REBUILD.md \$7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check\_md\_links, python -m tools.leak\_scan, git diff --check, and the full coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in

GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>

</heartbeat>

**528. assistant (2026-07-05T12:19:03.118Z)**

I'll use the code-review workflow for this heartbeat, then sync the repo before looking at PR state so I'm comparing against remote truth.

**529. assistant (2026-07-05T12:19:17.118Z)**

The checkout is already at the remote head. I'm checking open PRs, mergeability, heads, CI, and local branch state now.

**530. assistant (2026-07-05T12:19:34.605Z)**

#493 has moved to a new head (`c8f3031`) and is mergeable again, so this is no longer a quiet heartbeat. I'm going to treat it as a fresh review on top of current master `851f21d`; #490's head is unchanged from the blocked review.

**531. assistant (2026-07-05T12:19:46.554Z)**

I've got the PR metadata and prior review thread. The author rebased onto #492, CI is green, and the prior coverage blocker needs a new exact-base comparison against `851f21d`; I'm fetching the GitHub merge ref into a detached review worktree now.

**532. assistant (2026-07-05T12:20:36.883Z)**

The code review itself is looking contained: it adds `on\_tick` to the polling primitive, threads `on\_wait` through Cloud Run compute, and updates job progress with elapsed-time wording. I'm moving from inspection to the verified gate now, including a fresh base coverage run so the previous no-regression concern is settled cleanly.

**533. assistant (2026-07-05T12:20:48.947Z)**

I've got an exact-base worktree at `851f21d` and the PR merge-ref worktree at `0df7310`. Running the required non-suite gates first, then I'll run both coverage suites against the same command.

**534. assistant (2026-07-05T12:21:30.864Z)**

The fast gates are clean: lint, mypy, markdown links, leak scan, and whitespace. Now the slower bit: full coverage suite on both the PR merge ref and exact base, using the repo's `--cov-fail-under=70` command.

**535. assistant (2026-07-05T12:24:48.035Z)**

The full suite passed on both sides. The old coverage regression is resolved: exact base `851f21d` is 85.65%, PR merge ref `0df7310` is 85.66%. I'm doing one last head/CI re-check before posting the LGTM comment.

**536. assistant (2026-07-05T12:25:10.701Z)**

Posted the LGTM verification comment on #493. Since this changes the live review state, I'm refreshing the handoff/current-state so the next heartbeat starts from the right picture.

**537. assistant (2026-07-05T12:25:23.440Z)**

I'm updating only the live checkpoint sections, not rewriting the whole history. The important new fact is: #493 is verified-LGTM on `c8f3031`; #490 remains the lone engine blocker.

**538. assistant (2026-07-05T12:27:19.795Z)**

The handoff is refreshed. I'm doing a final sanity check for stale top-level status and making sure the main checkout stayed clean.

**539. assistant (2026-07-05T12:27:33.542Z)**

I'm cleaning up the two temporary review worktrees I created for the exact merge/base coverage comparison, then I'll leave the heartbeat with a notify because #493 materially changed state.

**540. assistant (2026-07-05T12:27:59.796Z)**

Posted verified LGTM on #493 after a fresh gate on `c8f3031` / merge ref `0df7310`. Coverage is now non-regressing: PR `85.66%` vs exact base `851f21d` at `85.65%`. #490 remains blocked; no merge performed.

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <decision>NOTIFY</decision>
 <message>#493 is now verified-LGTM and awaiting owner merge; #490 remains behavior-
blocked/LGTM-paused.</message>
</heartbeat>
```

**541. user (2026-07-05T12:28:27.659Z)**

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <current_time_iso>2026-07-05T12:28:27.611Z</current_time_iso>
 <instructions>
Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially
docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-
only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or
holistic final deploy/testing work. For each candidate PR not already reviewed by this thread,
apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads;
reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training,
python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full
coverage suite with the repo coverage floor; compare coverage against the exact base commit when
possible; examine security/correctness/performance/test completeness. Post LGTM only when no
blocking findings remain and the verified gate passes. If the PR is authored by the signed-in
GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If
there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an
owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do
not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful
checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy
verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof,
edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.
 </instructions>
</heartbeat>
```

**542. assistant (2026-07-05T12:29:07.767Z)**

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <decision>DONT_NOTIFY</decision>
 <message>No new PR head changes: #493 remains verified-LGTM awaiting owner merge, and #490
remains behavior-blocked/LGTM-paused.</message>
</heartbeat>
```

**543. user (2026-07-05T12:38:27.804Z)**

```
<heartbeat>
```

```
<automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
<current_time_iso>2026-07-05T12:38:27.762Z</current_time_iso>
<instructions>
Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially
docs/CONTROL_PLANE_DURABLE_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-
only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or
holistic final deploy/testing work. For each candidate PR not already reviewed by this thread,
apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads;
reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training,
python -m tools.check_md_links, python -m tools.leak_scan, git diff --check, and the full
coverage suite with the repo coverage floor; compare coverage against the exact base commit when
possible; examine security/correctness/performance/test completeness. Post LGTM only when no
blocking findings remain and the verified gate passes. If the PR is authored by the signed-in
GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If
there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an
owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do
not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful
checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy
verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof,
edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.
</instructions>
</heartbeat>
```

#### 544. assistant (2026-07-05T12:39:14.357Z)

#493 has merged to master and a new non-draft P3 PR (#494) appeared, so I'm shifting this heartbeat into a fresh verified review rather than staying quiet. I've already fast-forwarded master to `4cd36fd`; next I'm inspecting #494 against the P3 spec and then I'll run the full gate on the exact merge result.

#### 545. assistant (2026-07-05T12:39:25.487Z)

#494 is the new P3 recovery/read-path re-hydrate PR. It has already been refreshed onto merged #493, but GitHub checks are still running, so I'm going to review the exact merge ref locally while CI finishes.

#### 546. assistant (2026-07-05T12:39:53.920Z)

The PR is substantial but narrowly aimed: startup resume for md5-keyed running remote process jobs, `/api/query` read-path rehydrate, and carrying `video\_uri` in worker reports. I'm reading the changed control-flow paths now before I spend the ~3 minutes on the full coverage run.

#### 547. assistant (2026-07-05T12:40:09.925Z)

The first read surfaced the expected pieces; no obvious merge-conflict artifact. I'm going deeper on the recovery helper and the cache rehydrate source-path logic, since that's where a quiet correctness bug would live if there is one.

#### 548. assistant (2026-07-05T12:40:23.606Z)

The design choice is deliberately conservative: running md5-keyed process rows are not failed at startup, but they are also not re-dispatched. I'm checking whether that carve-out can accidentally keep a purely local process row alive, and whether read-path rehydrate is scoped only to md5-shaped IDs under the configured output bucket.

#### 549. assistant (2026-07-05T12:40:42.698Z)

One risk area is whether the resume sweep can mistake a local/forced process for an in-flight Cloud Run job just because it has an md5. I'm checking the submit metadata and force/compute fields against the recovery SQL before calling that a finding.

#### 550. assistant (2026-07-05T12:41:16.229Z)

I found a likely blocker around `force=true`: a forced reprocess row still has an md5, so the

startup carve-out treats it as resumable, but the resume poll keys only on `outputs/<md5>/report.json`. If an older successful report already exists, the restarted CP can mark the forced job done from stale cache while the new Cloud Run execution is still in flight. I'm reproducing that shape locally before posting anything.

**551. assistant (2026-07-05T12:41:35.195Z)**

The repro confirms it: a forced reprocess that was running before restart survives `recover\_stale\_jobs`, then immediately completes from an older cached report. I'm going to post that as a blocking PR comment and pause LGTM; no need to burn the full coverage suite until this behavior is corrected.

**552. assistant (2026-07-05T12:41:51.979Z)**

The blocker is posted on #494. I'm refreshing the top handoff/current-state so the next heartbeat knows #493 is merged and P3 is currently behavior-blocked on the `force=true` recovery case.

**553. assistant (2026-07-05T12:42:12.692Z)**

The current handoff still said #493 was open and P3 was a future build; I'm correcting that now so the next session doesn't chase a ghost.

**554. assistant (2026-07-05T12:42:41.279Z)**

I'm tightening the handoff's "you are here" section next: P6 is done now, and P3 exists but is waiting on an author fix rather than being a builder-lane future row.

**555. assistant (2026-07-05T12:43:00.411Z)**

One more pass on the quick-start bullets, then I'll clean up the temp review worktrees and report the heartbeat.

**556. assistant (2026-07-05T12:43:40.415Z)**

The handoff is now current. I'm removing only the two temp worktrees from this review and doing one final PR/head check so I don't report stale state.

**557. assistant (2026-07-05T12:44:03.639Z)**

#493 is merged to master as `4cd36fd`. I reviewed new P3 PR #494 and posted a blocker: `force=true` recovery can complete from a stale cached report after restart, so no LGTM yet.

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <decision>NOTIFY</decision>
 <message>#493 merged; #494 is behavior-blocked on forced reprocess restart recovery using
stale outputs, and #490 remains behavior-blocked.</message>
</heartbeat>
```

**558. user (2026-07-05T12:48:27.902Z)**

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <current_time_iso>2026-07-05T12:48:27.865Z</current_time_iso>
 <instructions>
```

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially docs/CONTROL\_PLANE\_DURABLE\_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check\_md\_links, python -m tools.leak\_scan, git diff --check, and the full

coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>  
</heartbeat>

**559. assistant (2026-07-05T12:49:07.436Z)**

#494 has a new head (`c824dfc`) after the blocker, so I'm treating this as a fresh P3 review. CI's long suite is still running, but the fast checks are green; I'm fetching the exact merge ref and checking the forced-reprocess fix first.

**560. assistant (2026-07-05T12:50:12.891Z)**

Using the code-review workflow for this heartbeat and picking up #494 at its new head; I'll verify the author's fix against the blocker before deciding whether it earns an LGTM.

**561. assistant (2026-07-05T12:50:37.257Z)**

The local checkout is fresh on `master` and clean. I'm checking the open PR set and #494's exact head/CI before recreating the merge locally.

**562. assistant (2026-07-05T12:50:43.285Z)**

#494's CI is now fully green at `c824dfc`, and the prior blocker has a claimed targeted fix. I'm moving to an exact merge-ref worktree so the local review matches what GitHub would merge.

**563. assistant (2026-07-05T12:51:05.390Z)**

The first fetch kept an old local PR merge ref because Git rejected the non-fast-forward ref update; good catch before testing. I'm replacing that temp merge worktree and force-updating only the local PR ref so the review uses `c824dfc`.

**564. assistant (2026-07-05T12:51:27.384Z)**

Now I have the real merge ref: `f22a40d` merging `c824dfc` into `4cd36fd`. I'm reading the changed recovery paths with the forced-run edge case in mind, plus the broader restart/resume behavior.

**565. assistant (2026-07-05T12:51:38.327Z)**

The forced-run blocker is excluded in both the recovery SQL and the resume function's defensive path. I'm widening the pass now: read-path rehydration, startup ordering, and tests, because restart fixes tend to hide edge cases in the glue.

**566. assistant (2026-07-05T12:51:55.402Z)**

A small but relevant seam: startup's row-list intentionally omits `params`, relying on the SQL filter to exclude forced rows; the defensive forced-row check still works for direct callers/tests. I'm checking whether the `remote job` classification can accidentally resume local-only work.

**567. assistant (2026-07-05T12:52:14.935Z)**

I found a sharper restart classification question: the code says `remote jobs`, but the SQL currently keys only on `running + video\_id`, not on whether the job was actually cloud-

dispatched. I'm tracing submit/cache order before deciding if that is a real blocker or just theoretical.

**568. assistant (2026-07-05T12:52:42.338Z)**

This looks real: a `compute="local"` row can be left in `running` and then completed from `outputs/<md5>` even though no remote worker exists. I'm going to reproduce that exact shape locally before I comment on the PR.

**569. assistant (2026-07-05T12:53:04.357Z)**

The repro confirms a blocker on the fixed head: a local override job is classified as resumable remote work and completes from the bucket cache. I'm grabbing tight line references and then I'll leave the PR paused with evidence.

**570. assistant (2026-07-05T12:53:31.343Z)**

I posted the blocker on #494 with the repro and a concrete predicate recommendation. I'm updating the living handoff so the next heartbeat does not accidentally treat `c824dfc` as pending-clean.

**571. assistant (2026-07-05T12:53:51.491Z)**

The handoff needs a surgical update rather than a rewrite: #494's prior `force=true` finding was addressed, but the new `compute="local"` misclassification is now the active blocker.

**572. assistant (2026-07-05T12:55:02.884Z)**

The memory files now point at the active #494 blocker and comment, not the superseded force finding. I'm cleaning up only the temp review worktrees I created, then I'll do a final status check.

**573. assistant (2026-07-05T12:55:26.911Z)**

The PR set is unchanged after the comment: #494 remains open at `c824dfc`, CI green but behavior-blocked; #490 remains the earlier P2 blocker. Main checkout is clean on `master`.

**574. assistant (2026-07-05T12:55:56.981Z)**

Posted a new blocker on #494: the `force=true` fix is good, but `compute="local"` jobs are still classified as remote-resumable and can complete from stale bucket output. No LGTM posted. Handoff/current-state updated.

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <decision>NOTIFY</decision>
 <message>#494 remains behavior-blocked after local repro; no LGTM posted, and handoff/current-state now point to the active compute-local recovery blocker.</message>
</heartbeat>
```

**575. user (2026-07-05T12:58:28.153Z)**

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <current_time_iso>2026-07-05T12:58:28.065Z</current_time_iso>
 <instructions>
```

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially docs/CONTROL\_PLANE\_DURABLE\_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check\_md\_links, python -m tools.leak\_scan, git diff --check, and the full



coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>  
</heartbeat>

**576. assistant (2026-07-05T12:58:44.608Z)**

Checking the monitor heartbeat now: I'll sync first, then only re-open review work if a PR head moved or a new candidate appeared.

**577. assistant (2026-07-05T12:58:59.014Z)**

Master is already current. I'm checking open PR heads, comments, review threads, and CI to see whether the blocker state changed.

**578. assistant (2026-07-05T12:59:13.245Z)**

There is one subtle change in status: #490's branch did not move, but its blocking dependency (#492) is now on master. I'm going to re-review #490 against the current merge result instead of leaving the paused LGTM stale.

**579. assistant (2026-07-05T12:59:19.188Z)**

I'm creating a fresh merge-ref worktree for #490 now. Because GitHub PR merge refs can move non-fast-forward as base changes, I'll force-update only the local `origin/pr/490/\*` refs.

**580. assistant (2026-07-05T12:59:44.797Z)**

GitHub's `pull/490/merge` ref is stale against `97feff4`, while local `origin/master` is already `4cd36fd`. I'm going to simulate the current merge in a separate worktree from `origin/master` so the review includes the #492/#493 base fixes.

**581. assistant (2026-07-05T13:00:07.893Z)**

The current-master simulated merge is clean, and it only needed an auto-merge in `backend/storage/gcs.py`. I'm checking the former 507 path now before spending the long full-suite time.

**582. assistant (2026-07-05T13:00:19.650Z)**

The diff still looks scoped to streaming upload staging. The current-base merge includes the #492 `jobs.submit\_process\_job` ordering, so I'm running focused regressions first, including the old 10 GiB/1 MiB cached-hit shape.

**583. assistant (2026-07-05T13:00:30.640Z)**

The focused upload file path is fine, but I used the wrong name for the #492 regression test. I'm locating the exact cached-hit test and rerunning it.

**584. assistant (2026-07-05T13:00:45.694Z)**

The upload-streaming regressions pass. I found the exact #492 cache-hit guard test and am running it with the paired miss/failure cases to make sure the old paused blocker is actually gone on the merged base.

**585. assistant (2026-07-05T13:01:01.998Z)**

The old blocker is fixed on the current-base merge: cache hit returns instant `done`, while true cache miss still 507s pre-claim. I'm starting the full verified gate now, including current-base coverage comparison.

**586. assistant (2026-07-05T13:01:20.530Z)**

I'm running the fast static gates and both full coverage suites now. This is the long part; I'll keep the final verdict tied to the actual outputs, not the PR description.

**587. assistant (2026-07-05T13:04:38.724Z)**

All gates are green and coverage is flat against current master (`85.66%` both sides). I'm doing the usual last head/CI check before posting a restored LGTM on #490.

**588. assistant (2026-07-05T13:04:58.547Z)**

#490 now has a restored verified LGTM comment. I'm updating the handoff/current-state so the next heartbeat knows #490 is ready for owner merge while #494 remains blocked.

**589. assistant (2026-07-05T13:06:36.849Z)**

The top-level current-state checkpoint is updated. I'm patching the handoff in the spots that named #490 as blocked.

**590. assistant (2026-07-05T13:07:14.856Z)**

The main handoff state now reflects #490 as LGTM. I'm updating the quick-start/priority lines too so there's no contradictory `blocked` note lower down.

**591. assistant (2026-07-05T13:07:54.118Z)**

The handoff is consistent now. I'm cleaning the temporary review worktrees and the short-lived local review branch, leaving the real repo/worktrees alone.

**592. assistant (2026-07-05T13:08:09.955Z)**

One more thing changed while I was gating #490: the existing `wt-p3-recovery` worktree now points at a newer commit. I'm checking GitHub again before I close this heartbeat, because #494 may already have a follow-up fix.

**593. assistant (2026-07-05T13:08:16.946Z)**

#490 appears to have merged while I was cleaning up: the main checkout is one commit behind and only #494 remains open. I'm pulling master, then I'll review the new #494 fix at `9c98b47`.

**594. assistant (2026-07-05T13:08:30.253Z)**

Master is now `6acddce` with #490 merged. I'm refreshing #494 metadata against that new base before I build the review worktree.

**595. assistant (2026-07-05T13:08:38.815Z)**

#494 has a new fix commit and CI is green. Since master moved to include #490, I'm reviewing a fresh current-base merge rather than the stale GitHub base ref.

**596. assistant (2026-07-05T13:09:04.197Z)**

The current-base merge for #494 is clean (`0595a9d` = `6acddce` + `9c98b47`). I'm reading the recovery predicate changes and the new regressions before running the long gate.

**597. assistant (2026-07-05T13:09:18.283Z)**

The revised predicate is doing the right split in code: SQL excludes explicit local/force, and the resume pass uses config to decide whether `compute IS NULL` was cloud-default. I'm checking the remaining read-path/worker/report glue, then I'll rerun the reproduced blockers and full gate.

**598. assistant (2026-07-05T13:09:46.286Z)**

The covered lowercase local path is fixed. I'm checking one normalization edge because the API lowercases `req.compute` when deciding execution, but the new recovery predicate compares the stored column differently.

**599. assistant (2026-07-05T13:10:08.436Z)**

That normalization edge reproduces the same stale-cache failure: `compute="LOCAL"` is executed as local by submit processing, but restart recovery treats it as resumable. I'm posting this as a blocker rather than burning the long suite.

**600. assistant (2026-07-05T13:10:28.103Z)**

I posted the #494 blocker with the uppercase `LOCAL` repro. I'm updating memory again: #490 is now merged on master, and #494 is blocked on compute normalization, not the previous lowercase path.

**601. assistant (2026-07-05T13:10:46.976Z)**

Current-state is refreshed. I'm adjusting the handoff header and quick-start so it says #490 merged and #494's active blocker is the normalization edge.

**602. assistant (2026-07-05T13:11:28.367Z)**

The handoff header and P-row narrative now match the true state. I'm cleaning up the remaining quick-start mentions.

**603. assistant (2026-07-05T13:12:05.002Z)**

Memory is now aligned: #490 merged, #494 blocked on normalization. I'll remove only the temporary #494 review worktrees and local review branch.

**604. assistant (2026-07-05T13:12:20.816Z)**

The working checkout is clean on current master `6acddce`; only the standing non-temp worktrees remain. I'm doing one final PR snapshot for the heartbeat response.

**605. assistant (2026-07-05T13:13:09.894Z)**

#490 is verified and merged on master at `6acddce`. I re-reviewed #494 on its new head and found one remaining blocker: uppercase/padded local compute values can still resume as remote jobs, so I posted the evidence on the PR and did not LGTM. Handoff/current-state are updated.

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <decision>NOTIFY</decision>
 <message>#490 merged; #494 is blocked on compute normalization after review, with evidence
 posted on the PR.</message>
</heartbeat>
```

**606. user (2026-07-05T13:13:58.341Z)**

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <current_time_iso>2026-07-05T13:13:58.290Z</current_time_iso>
 <instructions>
```

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially

docs/CONTROL\_PLANE\_DURABLE\_REBUILD.md \$7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check\_md\_links, python -m tools.leak\_scan, git diff --check, and the full coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>

</heartbeat>

**607. assistant (2026-07-05T13:14:12.980Z)**

I'm doing the next monitor pass: fresh pull first, then I'll check open PRs and only re-run the heavy gate if there's a new or updated candidate since the last blocker.

**608. assistant (2026-07-05T13:14:25.398Z)**

Pull is clean. I'm checking the live PR list and the project handoff now so I can distinguish new work to review from the already-posted #494 blocker.

**609. assistant (2026-07-05T13:14:37.394Z)**

Only #494 is open, and its head is still `9c98b47`. I'm checking the latest thread activity before deciding whether this is just waiting on the author or needs a fresh review pass.

**610. assistant (2026-07-05T13:14:55.572Z)**

No one has replied on #494 after the normalization blocker; it's waiting on an author push. I'm also giving the sibling P5b frontend PR a quick freshness check because it is part of the same P-row sequence.

**611. assistant (2026-07-05T13:15:10.603Z)**

#162 is still open, non-draft, CI-green, and I don't see a review from this thread. I'm switching into a proper review gate for it now: fresh merge result, diff/CI check, then the repo's frontend test/typecheck/coverage commands.

**612. assistant (2026-07-05T13:15:21.784Z)**

The #162 diff is intentionally narrow: it raises the ingest and compile polling ceilings from 10 to 60 minutes plus one test update. I'm pulling the workflow/package commands from `web/` now, then I'll run them against a clean PR merge worktree.

**613. assistant (2026-07-05T13:15:35.692Z)**

I'm going to review #162 on a merge commit, not the shared checkout, so any test artifacts stay isolated and the base comparison is honest.

**614. assistant (2026-07-05T13:15:53.828Z)**

The synthetic merge is clean (`0d3a88e` over `origin/master`), with only the intended three web files changed. I'm checking the actual merged code and then starting the local CI-equivalent commands.

**615. assistant (2026-07-05T13:16:03.989Z)**

Dependencies are pinned through `package-lock` and the local Node is newer than CI's Node 24, which is acceptable for this constants-only review but I'll still call it out if anything fails. Starting `npm ci` for both surfaces.

**616. assistant (2026-07-05T13:16:24.340Z)**

Installs are clean. I'm running the fast gates first, then I'll run the browser-backed CUJ/layout/all gates; those are the ones that catch rendering or timeout-copy regressions despite this being a small constants change.

**617. assistant (2026-07-05T13:16:59.485Z)**

The first shell-script parse check tripped on Windows path quoting, not on the scripts themselves. I'm rerunning it the CI way with relative paths, and continuing into the browser gates.

**618. assistant (2026-07-05T13:17:15.937Z)**

That parse check is still losing `\$f` while crossing PowerShell to bash, so I'm taking the boring route: three explicit relative `bash -n` calls. Also the coverage reporter is text-only here, so I'll capture the summary from command output/base rerun instead of looking for a JSON artifact.

**619. assistant (2026-07-05T13:18:14.931Z)**

The PR merge worktree is passing the web CUJ build/smoke/e2e and the real-browser layout guard. I'm finishing the remaining site render and accessibility checks, then I'll do the exact-base coverage comparison.

**620. assistant (2026-07-05T13:18:38.557Z)**

All PR-side CI-equivalent gates are green. I'm doing one tighter coverage-summary rerun and an exact-base coverage run so the LGTM can include the "no regression" check rather than leaning on the PR text.

**621. assistant (2026-07-05T13:18:57.893Z)**

The PR coverage is 94.79% statements/lines and 88.23% branches, well above floors. I'm measuring the exact base now; because the change is one constants-only frontend PR, the web coverage comparison is the relevant regression check.

**622. assistant (2026-07-05T13:19:45.469Z)**

One tiny wrinkle: the exact-base branch percentage is 88.24%, and the PR run reported 88.23% locally even though the change is constants/comments. I'm pulling JSON summaries from both runs so I can tell whether this is real coverage movement or V8/reporting noise before I stamp it.

**623. assistant (2026-07-05T13:20:21.218Z)**

The exact-base coverage comparison is clean when measured with the same reporter: PR and base both report 94.79% statements/lines, 88.23% branches, 89.40% functions. I don't see a blocking logic issue, so I'm posting the verified LGTM on #162.

**624. assistant (2026-07-05T13:20:36.783Z)**

I've posted the #162 verified LGTM. I'm refreshing the handoff/current-state now so the next heartbeat doesn't rediscover that review from scratch.

**625. assistant (2026-07-05T13:22:04.791Z)**

Handoff is updated. I'm cleaning up the temporary review/base worktrees now, leaving the shared

checkouts untouched.

**626. assistant (2026-07-05T13:22:30.199Z)**

One last quick poll before I close this heartbeat: if #494 moved while the #162 gate was running, I'll catch it instead of handing you stale state.

**627. assistant (2026-07-05T13:22:46.428Z)**

Fresh poll found #494 moved while I was finishing #162, head is now `d915444d` with CI green. That's exactly the blocker-fix shape, so I'm picking it up immediately instead of leaving it for the next heartbeat.

**628. assistant (2026-07-05T13:22:58.817Z)**

I'm going to reproduce #494 on a current-base merge again. Its GitHub base still points at pre-#490 `4cd36fd`, so I'm explicitly merging `d915444d` onto current `origin/master` (`6acddce`) before testing.

**629. assistant (2026-07-05T13:23:21.676Z)**

The fetch hit an old local `origin/pr/494/head` namespace wrinkle, so I'm verifying the merge worktree actually contains the new `d915444` normalization before trusting it.

**630. assistant (2026-07-05T13:23:31.091Z)**

Good catch: the first #494 worktree accidentally merged the stale local ref (`9c98b47`), not the new GitHub pull ref. I'm throwing that worktree away and fetching the pull head into a non-conflicting ref before continuing.

**631. assistant (2026-07-05T13:23:50.767Z)**

Now the merge is the right one (`14acc53`: `6acddce` + `d915444`). I'm running the restart-recovery regressions first, including the uppercase/padded compute case that blocked the prior head.

**632. assistant (2026-07-05T13:24:04.008Z)**

The recovery suite is green: 24 restart tests passed, including the round-3 normalization cases. One narrow worker-report test name from my memory was wrong, so I'm locating the actual test and continuing with the full gate.

**633. assistant (2026-07-05T13:24:23.215Z)**

The focused restart and compute-backend tests are green. I'm doing a quick logic pass over the resume predicate and dispatch semantics before burning the full suite, because this PR lives right on a restart/spend boundary.

**634. assistant (2026-07-05T13:24:34.125Z)**

The logic pass looks consistent with the execution path now: SQL excludes normalized local values, resume lowercases request compute, NULL rows only resume when server default is Cloud Run, and explicit cloud override still resumes when the cloud wiring exists. Full local gate is next.

**635. assistant (2026-07-05T13:25:19.547Z)**

Static gates are clean. I'm running the full coverage suite on the current-base #494 merge now; if it holds, I'll measure the exact base with the same command for the coverage comparison.

**636. assistant (2026-07-05T13:28:39.230Z)**

Full #494 merge suite passed: 1725 passed, 10 skipped, coverage 85.71%. Now I'm measuring exact

base `6acddce` with the same command so the coverage comparison is apples-to-apples.

**637. assistant (2026-07-05T13:31:57.917Z)**

Exact-base comparison is good: base `6acddce` was 1701 passed, coverage 85.66%; #494 current-base merge was 1725 passed, coverage 85.71%. I'm doing one final PR-head check, then I'll post the verified LGTM if it hasn't moved.

**638. assistant (2026-07-05T13:32:03.828Z)**

#494 stayed on `d915444` with all GitHub checks green. I'm posting the verified LGTM now.

**639. assistant (2026-07-05T13:32:26.126Z)**

#494 has the verified LGTM now. I'm refreshing state again because the world moved under us: #162 is already merged, and #494 is now in ?await owner merge? instead of blocked.

**640. assistant (2026-07-05T13:32:39.607Z)**

State is changing quickly, in the good way. I'm updating the handoff to: #162 merged as `90d1d90`, #494 verified-LGTM on `d915444` and awaiting explicit owner merge.

**641. assistant (2026-07-05T13:33:08.660Z)**

I found the handoff already had a newer sibling-session checkpoint mentioning #495 and the merge-on-LGTM sweep. I'm not overwriting that; I'm adding our verified #494 re-gate result above it so the record stays honest.

**642. assistant (2026-07-05T13:33:28.850Z)**

Now I'm checking the session handoff top before patching it, because it may also have newer sibling-session state and I want to fold in only the delta I verified.

**643. assistant (2026-07-05T13:33:43.443Z)**

The handoff had both the new ?merge-on-LGTM? top note and stale lower sections from before #494/#162 moved. I'm reconciling those specific stale bits now.

**644. assistant (2026-07-05T13:34:21.382Z)**

The top and project narrative are corrected. I'm cleaning the stale quick-start lines too, since those are what future sessions skim first.

**645. assistant (2026-07-05T13:35:01.795Z)**

One more thing surfaced in the updated handoff: #495 is open as P8. I'm polling the live PR list now; if #495 is non-draft and still unreviewed, I'll take it through the same gate rather than leaving a gap before P9.

**646. assistant (2026-07-05T13:35:15.090Z)**

#495 is non-draft, CI-green, and unreviewed. It's mostly vendored SPA assets plus \$7 tracker updates, but it still gets the real gate: current-base merge, asset/version sanity checks, docs checks, and full suite.

**647. assistant (2026-07-05T13:35:42.903Z)**

The #495 merge result is `188462e` over `6acddce`. I'm checking that the vendored manifest points at the actual khelsutra merge and that the docs row updates don't overclaim P9.

**648. assistant (2026-07-05T13:37:01.270Z)**

I'm picking up the review monitor from the handoff: #494 has a verified LGTM, and #495 is the

live P8 candidate. I'll resync before reading anything further, then finish the #495 vendored-SPA verification and full gate.

**649. assistant (2026-07-05T13:37:20.976Z)**

Master moved while I was in the middle of the monitor run: #494 has now landed, and #495 rebased/updated onto that merged P3 commit. I'm throwing away the stale mental model for #495 and recreating the review from the new remote base/head pair.

**650. assistant (2026-07-05T13:37:35.323Z)**

The PR has two commits now: the original P8 vendoring commit plus a follow-up that flips P3 to done after #494 merged. I'm going to verify the vendored assets against a fresh build of `khelsutra` at `90d1d90`, not just trust the manifest.

**651. assistant (2026-07-05T13:37:53.645Z)**

The synthetic merge for #495 is clean on top of the new master. I'm reading the exact docs/version/script touchpoints now, then I'll run the reproduction build and gates.

**652. assistant (2026-07-05T13:38:14.064Z)**

A quick read shows the manifest pins `90d1d90` and the docs now mark P3 complete, which matches the remote state after #494 merged. Next is the higher-signal check: rebuild the SPA from that exact commit and compare every vendored file/hash.

**653. assistant (2026-07-05T13:38:58.305Z)**

The source SPA build at `90d1d90` reproduces the same hashed JS/CSS filenames that #495 vendors. I'm doing byte-level comparison now, including the manifest content hash, so the review is pinned to actual artifacts rather than naming coincidence.

**654. assistant (2026-07-05T13:39:26.942Z)**

The payload bytes match, but my first PowerShell reimplementations of the bundle hash didn't reproduce the manifest value. That smells like my hash-recipe emulation, so I'm rerunning the exact `find | sha256sum | sha256sum` pipeline the bundling script uses before treating it as a finding.

**655. assistant (2026-07-05T13:39:40.903Z)**

That `bash` is WSL-style, so `/c/...` was the wrong path form. I'm rerunning with `/mnt/c/...`; the earlier byte comparison itself remains valid.

**656. assistant (2026-07-05T13:40:14.493Z)**

This surfaced a real-looking review concern: the vendored payload bytes match the SPA build, but `ui\_version.json.content\_hash` does not match the bundle script's own current hash recipe over those files. I'm checking whether that field is asserted anywhere and whether rerunning the script in a throwaway worktree reproduces the mismatch before I comment.

**657. assistant (2026-07-05T13:41:06.618Z)**

I'm going one step further before filing it: in a detached throwaway worktree, I'll rerun `scripts/bundle\_ui.sh` against the already-built `90d1d90` checkout. If the only meaningful delta is the manifest hash moving to the recomputed value, that's a crisp blocker and an easy fix for the PR author.

**658. assistant (2026-07-05T13:41:33.748Z)**

I've got a blocker for #495: the snapshot itself is right, but the generated manifest hash is not. I'm going to post that as a PR comment with the reproduction command and leave the PR un-LGTM'd until the author updates it.



**659. assistant (2026-07-05T13:41:56.853Z)**

#495 now has a blocking review comment with the exact reproduction. I'm refreshing the handoff/current-state so the next heartbeat starts from P8 waiting on manifest hash fix, then I'll clean up only the throwaway worktrees I created for this review.

**660. assistant (2026-07-05T13:42:12.124Z)**

There are repo-local and synced memory copies; the live handoff for this project is the repo root `SESSION\_HANDOFF.md`, with broader memory mirrored under OneDrive. I'm checking the current text before patching so I don't overwrite sibling-session notes.

**661. assistant (2026-07-05T13:42:41.570Z)**

I'm making the memory update surgical: add the new checkpoint up top, and rewrite only the handoff lines that were stale after #494 merged and #495 got the hash blocker.

**662. assistant (2026-07-05T13:43:37.248Z)**

The handoff now records #494 as merged and #495 as blocked on a specific provenance issue, not a vague 'needs review.' I'm updating the next-session grab/icebreaker to point directly at the re-review path.

**663. assistant (2026-07-05T13:44:22.155Z)**

Memory is refreshed. I'm cleaning up only the review worktrees I created for this heartbeat, with path checks first so I don't touch shared author worktrees like `wt-p8-vendor`.

**664. assistant (2026-07-05T13:44:50.573Z)**

Cleanup is done; the shared engine checkout is still on master and the author's `wt-p8-vendor` worktree is untouched. I'm doing one last poll of #495 before I decide whether this heartbeat needs a user-facing notification.

**665. assistant (2026-07-05T13:45:06.019Z)**

#494 has merged into master as `f6864f1`.

I reviewed #495 head `6644736`: the re-vendored SPA payload matches a fresh `khelsutra@90d1d90` build byte-for-byte, but `backend/ui/ui\_version.json.content\_hash` is stale/wrong (`a6fd...` committed vs `364da069...` recomputed). I posted the blocker here: [#495 comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/495#issuecomment-4886248799). No LGTM until that manifest provenance is fixed and the full gate is rerun.

Handoff/current-state are updated, and I cleaned up only my temporary review worktrees.

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <decision>NOTIFY</decision>
 <message>#494 merged; #495 is blocked on a stale ui_version content_hash despite matching SPA
 payload bytes, so no LGTM until the manifest is fixed.</message>
</heartbeat>
```

**666. user (2026-07-05T13:45:28.819Z)**

```
<heartbeat>
 <automation_id>khelsutra-cp-rebuild-pr-reviewer</automation_id>
 <current_time_iso>2026-07-05T13:45:28.763Z</current_time_iso>
 <instructions>
Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially
docs/CONTROL_PLANE_DURABLE_REBUILD.md $7 rows. On each run: sync the repo with git pull --ff-
only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or
holistic final deploy/testing work. For each candidate PR not already reviewed by this thread,
```

apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check\_md\_links, python -m tools.leak\_scan, git diff --check, and the full coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>

</heartbeat>

#### 667. assistant (2026-07-05T13:45:42.854Z)

Using the code-review workflow again, since this heartbeat is specifically a PR gate. I'm resyncing first and checking whether #495 has moved since the manifest-hash blocker.

#### 668. assistant (2026-07-05T13:45:54.035Z)

<heartbeat>

<automation\_id>khelsutra-cp-rebuild-pr-reviewer</automation\_id>

<decision>DONT\_NOTIFY</decision>

<message>#495 is unchanged since the manifest-hash blocker comment; CI is green but LGTM remains held until the head updates with a corrected ui\_version content\_hash.</message>

</heartbeat>

#### 669. user (2026-07-05T13:55:29.002Z)

<heartbeat>

<automation\_id>khelsutra-cp-rebuild-pr-reviewer</automation\_id>

<current\_time\_iso>2026-07-05T13:55:28.958Z</current\_time\_iso>

<instructions>

Monitor Khelsutra/badminton-highlight-indexer for incoming CP-rebuild spec PRs P1-P9, especially docs/CONTROL\_PLANE\_DURABLE\_REBUILD.md §7 rows. On each run: sync the repo with git pull --ff-only before reading, list open PRs, and identify non-draft PRs for the P-row sequence or holistic final deploy/testing work. For each candidate PR not already reviewed by this thread, apply the full verified-LGTM gate from AGENTS.md: inspect PR metadata/diff/CI/review threads; reproduce locally from a fresh remote base/worktree; run ruff check ., mypy backend training, python -m tools.check\_md\_links, python -m tools.leak\_scan, git diff --check, and the full coverage suite with the repo coverage floor; compare coverage against the exact base commit when possible; examine security/correctness/performance/test completeness. Post LGTM only when no blocking findings remain and the verified gate passes. If the PR is authored by the signed-in GitHub user and formal approval is blocked, post a clear LGTM/verification comment instead. If there are findings, comment on the PR with actionable evidence and do not LGTM. If there is an owner/product/legal/spend/deploy judgment call, pause and ask Avi to arbitrate in the thread. Do not merge unless Avi explicitly asks. Keep session handoff/current-state updated at meaningful checkpoints, and make sure final P9 validation is holistic and rigorous: deploy/redeploy verification, browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth posture, generated manifest pins, SPA vendored-version proof, and runlog/docs updates.

</instructions>

</heartbeat>

#### 670. assistant (2026-07-05T13:55:53.547Z)

#495 moved to `9af45b2` and CI is green, so the blocker may be fixed. I'm switching from polling to a full re-review: fresh merge worktree, manifest/hash verification, then the repo's required static and coverage gates.

#### 671. assistant (2026-07-05T13:56:09.747Z)

The author fixed the blocker in `9af45b2`: manifest changed, `bundle\_ui.sh` now normalizes the MSYS `sha256sum` marker, and `tests/test\_ui\_bundle.py` gained a drift guard. I'm reviewing that fresh head from `origin/master`, not the shared author worktree.

**672. assistant (2026-07-05T13:56:22.326Z)**

The synthetic merge is clean. I'm checking the exact diff now, then I'll run the manifest recipe and focused bundle tests before burning the full suite time.

**673. assistant (2026-07-05T13:56:56.191Z)**

The new test mirrors the hash recipe in Python and looks aimed exactly at the prior failure. I'm rebuilding the SPA from the pinned `khelsutra` commit and independently recomputing the hash now, so the manifest proof is not just ?the new test says so.?

**674. assistant (2026-07-05T13:57:33.687Z)**

The pinned SPA build reproduced cleanly at `90d1d90`; its generated asset names match the PR. I'm doing the byte-for-byte payload comparison and checking the cross-platform hash script change next.

**675. assistant (2026-07-05T13:57:55.202Z)**

The payload comparison passed. One reproducibility check hit a WSL/Git-on-Windows path quirk after running the bundle script inside the review worktree, so I'm checking the resulting diff and restoring the temp worktree back to the PR merge before continuing. This is only my disposable review worktree.

**676. assistant (2026-07-05T13:57:59.636Z)**

The only modifications are from my failed WSL-path repro attempt (`spa\_repo`/`spa\_commit`/`built\_at`), not from the PR. I'm reverting those two temp-worktree files to the reviewed merge commit and then continuing the gate.

**677. assistant (2026-07-05T13:58:09.482Z)**

The review worktree is clean again. The manifest hash and focused bundle test are good, so I'm moving into the required repo gate: diff-check, ruff, mypy, doc links, leak scan, and then full coverage versus exact base.

**678. assistant (2026-07-05T13:59:11.287Z)**

Static gates are clean. Because #495 changes a shell script too, I'm adding a shell parse check, then I'll run the full coverage suite on the PR merge and exact base.

**679. assistant (2026-07-05T13:59:18.477Z)**

Everything remains clean after the focused checks. I'm starting the PR full suite now; after that I'll measure exact-base coverage at `f6864f1` for the regression comparison.

**680. assistant (2026-07-05T14:02:45.944Z)**

PR full suite is green: 1727 passed, 10 skipped, coverage 85.71%, above the 70% floor. I'm measuring the exact base commit now so I can confirm no coverage regression.

**681. assistant (2026-07-05T14:02:54.724Z)**

That base-suite command tried to start in the not-yet-created worktree path. Nothing changed; I'm rerunning it from the workspace root so the worktree can be created first.

**682. assistant (2026-07-05T14:06:14.604Z)**

Exact-base coverage is also 85.71%, so there's no coverage regression; #495 adds two bundle-

guard tests, which accounts for 1727 vs 1725 tests. I'm doing a final head/CI/doc-status check before posting the review result.

**683. assistant (2026-07-05T14:06:26.077Z)**

Final remote state is stable at `9af45b2`, all GitHub checks are green, and the \$7 tracker now has P3 done with P8 still in-this-PR. I'm posting a verified LGTM comment rather than a formal approval because the PR is authored by the signed-in owner account.

**684. assistant (2026-07-05T14:06:38.654Z)**

#495 now has a verified LGTM comment with the full gate results. I'm updating the handoff/current-state from ?blocked? to ?verified-LGTM, awaiting merge/next explicit owner action,? then I'll remove the disposable review worktrees.

**685. assistant (2026-07-05T14:07:11.042Z)**

Current-state is refreshed. I'm patching the living handoff in the same narrow way: #495 is now verified-LGTM, and P9 remains next after #495 lands.

**686. assistant (2026-07-05T14:08:09.764Z)**

One handoff patch missed because that long paragraph has drifted slightly. I'm locating the exact text and will patch against what's on disk.

**687. assistant (2026-07-05T14:08:17.376Z)**

The next-session text picked up a separate #483 warm-backlog update, so I'm preserving that while changing just the stale #495 part.

**688. assistant (2026-07-05T14:08:28.585Z)**

Now I have the exact current lines; applying the final handoff correction.

**689. assistant (2026-07-05T14:09:00.149Z)**

Handoff is updated. I'm cleaning the disposable #495 review/base and SPA-build worktrees now, leaving the author worktree and shared checkouts alone.

**690. assistant (2026-07-05T14:09:24.368Z)**

One more change landed while I was cleaning up: the engine checkout is now behind origin by one commit. I'm pulling and checking whether #495 was merged so the handoff reflects shipped state, not just review state.

**691. assistant (2026-07-05T14:09:34.689Z)**

#495 has now merged as `257f469`, and there are no open engine PRs. I'm recording that shipped state in memory; the next meaningful monitor target is P9 deploy/redeploy validation work.

**692. assistant (2026-07-05T14:09:45.867Z)**

Current-state now marks #495 as merged. I'm making the same merge-state correction in the session handoff.

**693. assistant (2026-07-05T14:10:20.623Z)**

The top of the handoff is updated to ?P8 merged.? I'm checking the bottom quick-start text too, because that often carries stale PR instructions if we don't tidy it.

**694. assistant (2026-07-05T14:10:35.510Z)**

The bottom quick-start still says ?watch #495,? so I'm replacing that with ?P9 now.?

**695. assistant (2026-07-05T14:10:40.407Z)**

I've got the exact stale lines now; one final patch.

**696. assistant (2026-07-05T14:11:16.699Z)**

The handoff quick-start now points straight at P9. I'm doing a final sanity check: open PRs, repo status, and leftover worktrees.

**697. assistant (2026-07-05T14:11:30.667Z)**

#495 was fixed, fully re-gated, LGTM?d, and has now merged as `257f469`. There are no open engine PRs; P9 is the next active CP-rebuild item.

<heartbeat>

<automation\_id>khelsutra-cp-rebuild-pr-reviewer</automation\_id>

<decision>NOTIFY</decision>

<message>#495/P8 merged after verified LGTM; all P1-P8 rows needed for the rebuild are landed, so the next checkpoint is P9 holistic rebuild/redeploy validation.</message>

</heartbeat>